

Computer Organization and Operating System

UAMPC0502

Credit: 2

ISE1	10
MSE	30
ISE2	10
ESE	50

Overview

1. Introduction to Computer Organization
2. Introduction to Operating Systems- IPC,Synchronization
3. Deadlocks & File Management
4. Memory Management

	Story	Real-Life Scenario	OS Concepts	Computer Organization Concepts	Why It's Interesting
1	Ever wonder how Netflix can stream to millions of users without crashing or lagging?	Netflix streams to millions with no lag	Process scheduling, thread management, concurrency	Multi-core CPUs, cache hierarchy, instruction execution	Shows how OS + CPU design make real-time video possible
2	When you say "Hey Siri," your phone wakes up from a low-power state, loads the voice recognition engine, and responds — all within milliseconds.	"Hey Siri" triggers real-time voice recognition	Interrupts, process states, I/O management	ALUs, control units, low-power CPU states	Demonstrates efficiency of hardware-software collaboration
3	Tesla Model 3 processes data from dozens of sensors in real-time. It needs to decide within milliseconds whether to brake, accelerate, or turn.	Robots in warehouses make constant decisions	Process/thread scheduling, IPC, synchronization	DMA, CPU time-sharing, system buses	Business-critical — faster systems = higher profits
4	Smartphones Stay Cool	Phones manage performance without overheating	Task scheduling, power-aware OS features	ARM architecture, pipeline efficiency, instruction sets	How smartphones balance power, speed, and temperature

Unit – I

Introduction to Computer Organization

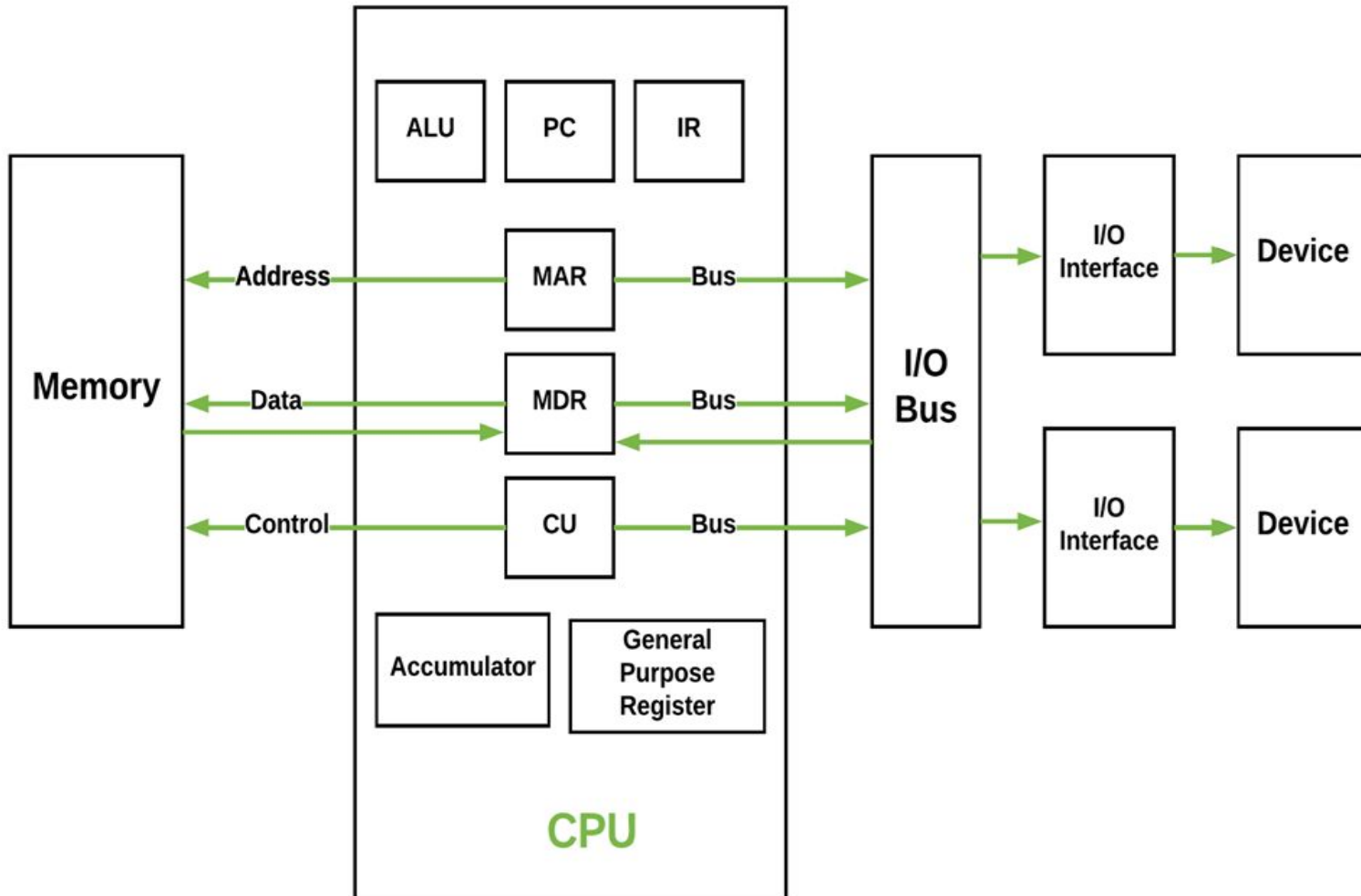
Outline

- Introduction to RISC and CISC architectures,
- ALU and control unit,
- Hardwired vs. Microprogrammed control,
- Pipelining and its performance,
- Memory organization and types,
- Cache memory: Mapping techniques and replacement policies

Why Computer Organization?

- Focuses on the internal structure and functional behavior of a computer system.
- It explains how different components (CPU, memory, I/O, buses) work together to execute instructions.
- Design better programs, including system software such as compilers, operating systems, and device drivers.
- Optimize program behaviour.
- Evaluate (benchmark) computer system performance.
- Understand time, space, and price tradeoffs.

Basic Computer Organization

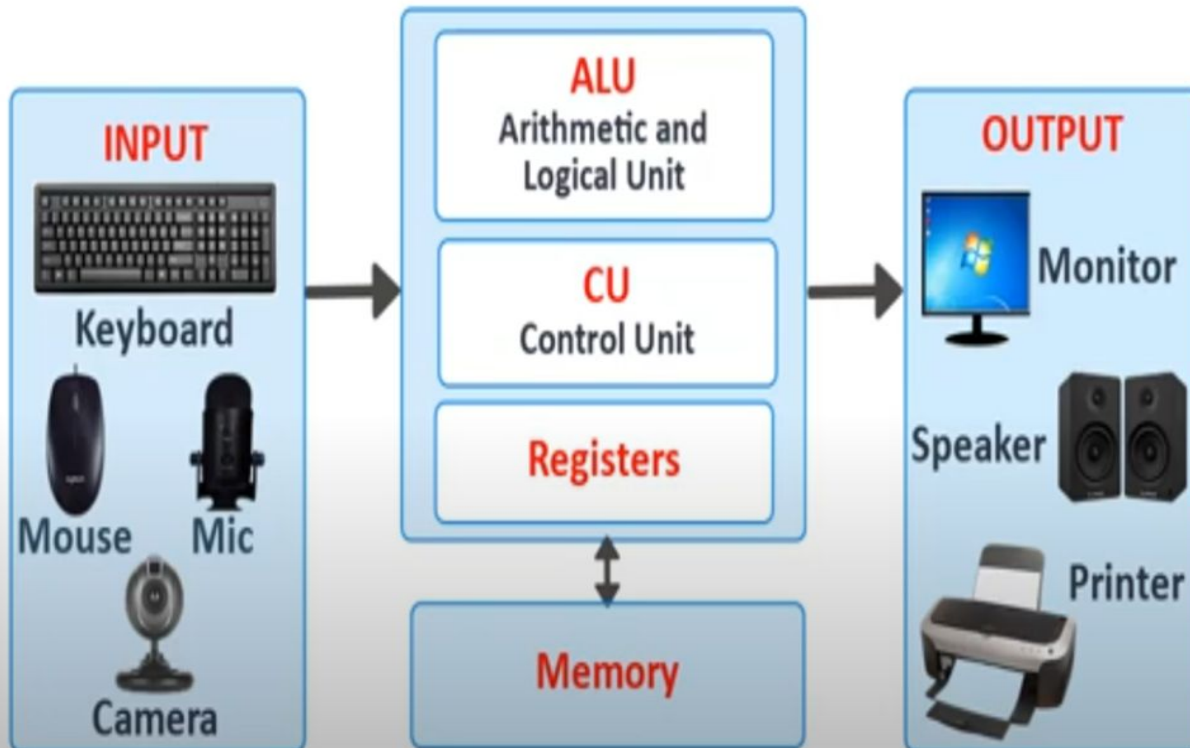


- Control Unit
- Arithmetic and Logic Unit (ALU)
- Main Memory Unit (Registers) -
 - a. Accumulator
 - b. Program Counter (PC)
 - c. Memory Address Register (MAR)
 - d. Memory Data Register (MDR)
 - e. Current Instruction Register (CIR)
 - f. Instruction Buffer Register (IBR)
- Input/Output Devices
- Buses
 - Data Bus
 - Address Bus
 - Control Bus

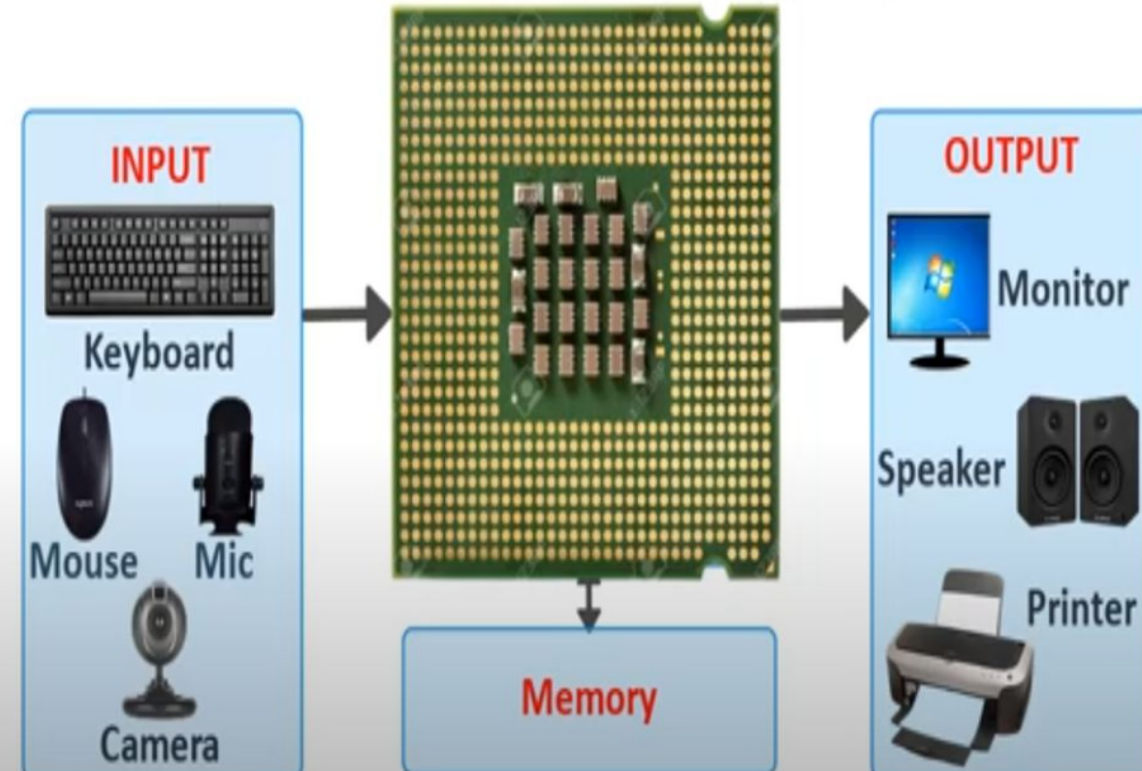
Microprocessor

A microprocessor is an integrated circuit that incorporates the central processing unit of a computer on a single chip. It takes binary input from devices, processes it according to instructions stored in memory, and provides output.

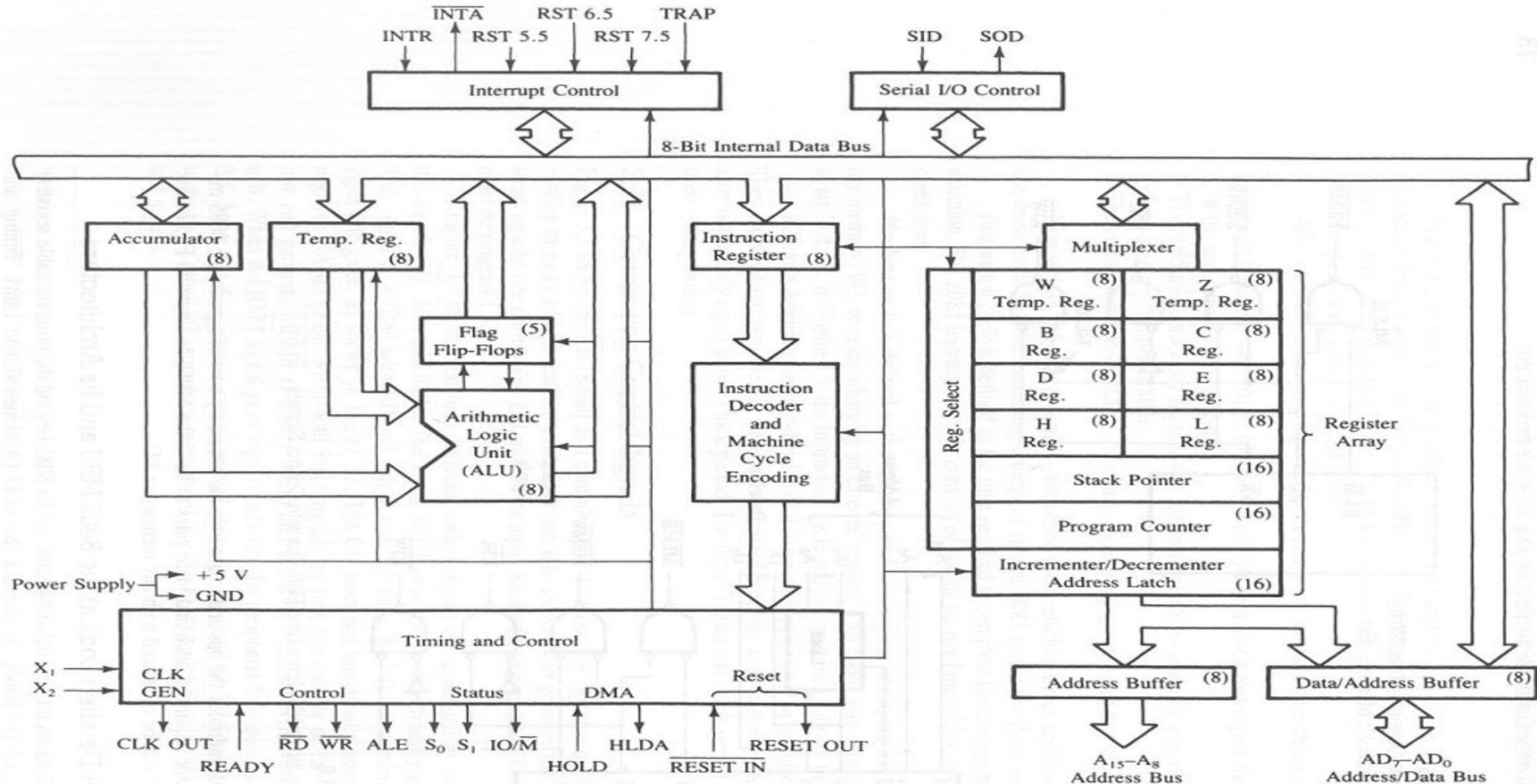
Central Processing Unit (CPU)



Central Processing Unit (CPU)



Architecture of 8085



Addressing mode

- It refers to the way in which the operand of an instruction is specified or accessed in a computer's memory.
- It defines how the effective address of the operand is calculated and where it is located.
- Addressing modes provide flexibility in data access, allowing for more efficient memory usage and instruction execution.
- Addressing Modes in computer architecture are further divided into the following parts:
 1. Implied/ Implicit Addressing Mode
 2. Immediate Addressing Mode
 3. Register Direct Addressing Mode
 4. Register Indirect Addressing Mode
 5. Auto-Increment Addressing Mode
 6. Auto-Decrement Addressing Mode
 7. Direct Addressing Mode
 8. Indirect Addressing Mode
 9. Displacement Addressing Mode
 10. Relative Addressing Mode
 11. Indexed Addressing Mode
 12. Base Register Addressing Mode
 13. Stack Addressing Mode

RISC vs. CISC Architectures

- **RISC (Reduced Instruction Set Computer):**

- Simple, uniform instructions
- Fewer addressing modes
- High performance via pipelining
- Example: ARM, MIPS

- **CISC (Complex Instruction Set Computer):**

- Rich set of complex instructions
- More addressing modes
- Emphasis on hardware simplicity for compiler
- Example: Intel x86

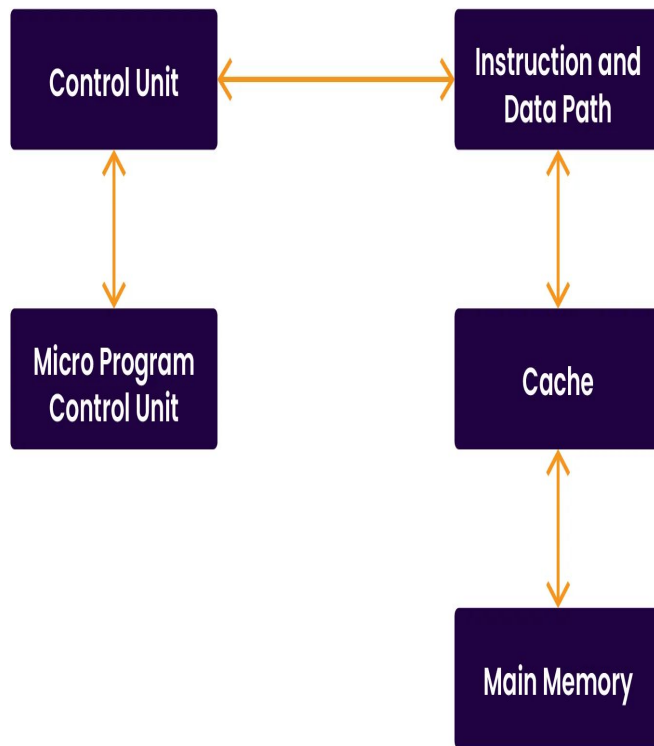
Introduction: CISC

- CISC means Complex Instruction Set Computer chips that are easy to program and which make efficient use of memory. Since the earliest machines were programmed in assembly language and memory was slow and expensive, the CISC philosophy was commonly implemented in large computers as the PDP-11 and the DECsystem 10 and 20 machines.
- Most common microprocessor designs such as the Intel 80x86 and Motorola 68K series followed the CISC philosophy.
- CISC was developed to make **compiler development simpler**. It shifts most of the burden of generating machine instructions to the processor.
- For example, instead of having to make a compiler write long machine instructions to calculate a square-root, a CISC processor would have a built-in ability to do this.

CISC Attributes

CISC instructions sets have some common characteristics:

- A **2-operand format**, where instructions have a source and a destination. Register to register, register to memory, and memory to register commands.
- Variable length instructions where the length often varies according to the addressing mode.
- Instructions which require **multiple clock cycles** to execute.
- E.g. Pentium is considered a modern CISC processor



Main objective of a CISC processor is to **minimize** the program size by **reducing** the **number of instructions** in a program.

This is done by 'embedding some of the low-level instructions in a single complex instruction'.

When decoded this instruction generates several microinstructions to execute.

If a program/software is getting simplified then the hardware has to get on work and must be able to perform the complex tasks. That's why a CISC processor has complex hardware.

Microprogram control unit that uses a series of microinstructions of the microprogram stored in the "control memory" of the *microprogram control unit* and generate the **control signals**.

The **control units** access the **control signals** produced by the *microprogram control unit* & operate the functioning of processors hardware.

Instruction and data path fetches the opcode and operands of the instructions from the memory.

Cache and **main memory** is the location where the program instructions and operands are stored.

Most CISC hardware architectures have several characteristics in common:

- Complex instruction-decoding logic, driven by the need for a single instruction to support multiple addressing modes.
- A small number of general purpose registers. This is the direct result of having instructions which can operate directly on memory and the limited amount of chip space not dedicated to instruction decoding, execution, and microcode storage.
- Several special purpose registers. Many CISC designs set special registers for the stack pointer, interrupt handling, and so on.
- A 'Condition code' register which is set as a side-effect of most instructions. This register reflects whether the result of the last operation is less than, equal to, or greater than zero and records if certain error conditions occur.

At the time of their initial development, CISC machines used available technologies to optimize computer performance.

- **Microprogramming** is as easy as assembly language to implement, and much less expensive than hardwiring a control unit.
- The ease of microcoding new instructions allowed designers to make CISC machines upwardly compatible: a new computer could run the same programs as earlier computers because the new computer would contain a superset of the instructions of the earlier computers.
- As each instruction became more capable, fewer instructions could be used to implement a given task. This made more efficient use of the relatively slow main memory.
- Because microprogram instruction sets can be written to match the constructs of high-level languages, the compiler does not have to be as complicated.

Complex Instruction Set Computer (CISC) Characteristics

Major characteristics of a CISC architecture

- A large number of instructions - typically from 100 to 250 instruction
- Some instructions that perform specialized tasks and are used infrequently
- A large variety of addressing modes - typically from 5 to 20 different modes
- Variable-length instruction formats
- Instructions that manipulate operands in memory (RISC in register)

Example

A = B + C

RISC

Load R2, B

Load R3, C

Add R4, R2, R3

Store R4, A

CISC

Move A, B

Add A, C

What is RISC?

- RISC, or Reduced Instruction Set Computer is a type of microprocessor architecture that utilizes a small, highly-optimized set of instructions, rather than a more specialized set of instructions often found in other types of architectures.
- History: The first RISC projects came from IBM, Stanford, and UC-Berkeley in the late 70s and early 80s. The IBM 801, Stanford MIPS, and Berkeley RISC 1 and 2 were all designed with a similar philosophy which has become known as RISC.

Certain design features have been characteristic of most RISC processors:

- **One cycle execution time:** RISC processors have a CPI (clock per instruction) of one cycle. This is due to the optimization of each instruction on the CPU.
- **Pipelining:** a technique that allows for simultaneous execution of parts, or stages, of instructions to more efficiently process instructions.
- **Large number of registers:** the RISC design philosophy generally incorporates a larger number of registers to prevent in large amounts of interactions with memory

Reduced Instruction Set Computer (RISC)

- Major characteristics of a RISC architecture
 1. Relatively few instructions
 2. Relatively few addressing modes
 3. Memory access limited to load and store instruction
 4. All operations done within the registers of the CPU
 5. Fixed-length, easily decoded instruction format
 6. Single-cycle instruction execution
 7. Hardwired rather than microprogrammed control

RISC processor is implemented using the **hardwired control unit**.

The hardwired control unit produces **control signals** which regulate the working of processors hardware.

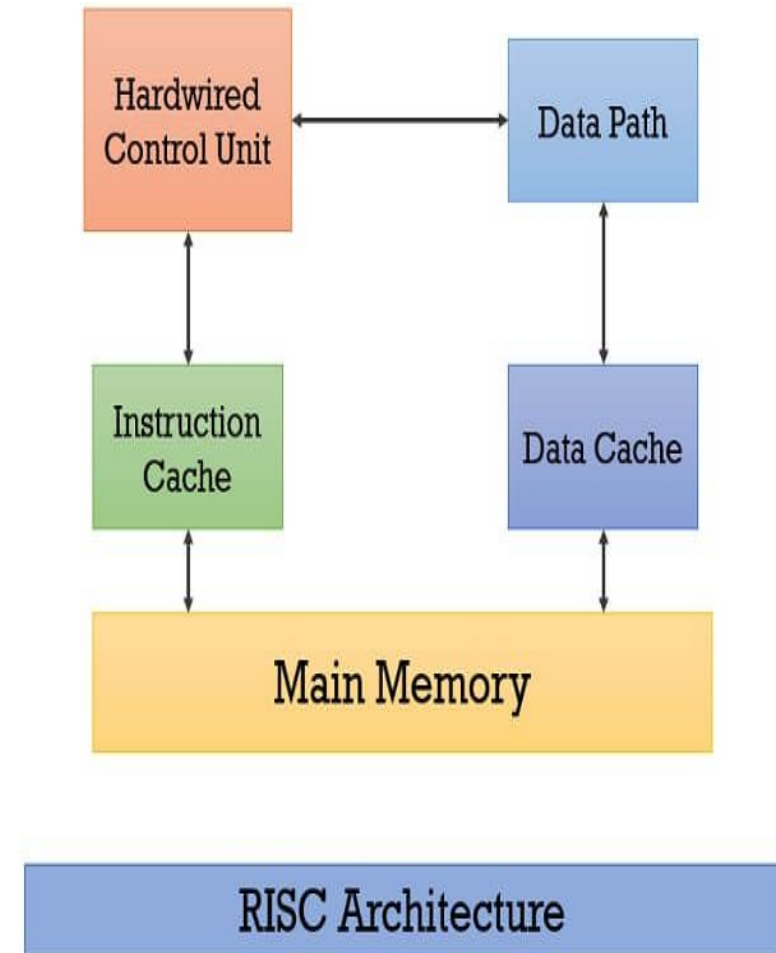
RISC architecture emphasizes on using the **registers** rather than memory.

This is because the registers are the 'fastest' available memory source.

The registers are physically small and are placed on the same chip where the ALU and the control unit are placed on the processor.

The RISC instructions **operate** on the operands present in **processor's registers**.

All instructions in RISC are simple and execute one instruction per cycle. So, here the **instructions are hardwired** and there is no need for control store. For each operation, we will have as defined hardwire. Making an instruction hardwired is making a function or operation in instruction **permanent** using connected circuits.



- RISC Instruction

- Only use LOAD and STORE instruction when communicating between memory and CPU
- All other instructions are executed within the registers of the CPU without referring to memory

Program to evaluate $X = (A + B) * (C + D)$

LOAD	R1, A	$R1 \leftarrow M[A]$
LOAD	R2, B	$R2 \leftarrow M[B]$
LOAD	R3, C	$R3 \leftarrow M[C]$
LOAD	R4, D	$R4 \leftarrow M[D]$
ADD	R1, R1, R2	$R1 \leftarrow R1 + R2$
ADD	R3, R3, R4	$R3 \leftarrow R3 + R4$
MUL	R1, R1, R3	$R1 \leftarrow R1 * R3$
STORE	X, R1	$M[X] \leftarrow R1$

- Load instruction transfers the operand from memory to CPU Register.

- Add and Multiply operations are executed with data in the registers without accessing the memory.

- Result is then stored in the memory with store information.

- Other characteristics of a RISC architecture:

1. A relatively large number of registers in the processor unit
2. Use of overlapped register windows to speed-up procedure call and return
3. Efficient instruction pipeline
4. Compiler support for efficient translation of high-level language programs into machine language programs

CISC vs RISC

CISC	RISC
Emphasis on hardware	Emphasis on software
Uses microprogrammed control unit	Uses only Hardwired control unit
Includes multi-clock complex instructions	Single-clock, reduced instruction only
Memory-to-memory: "LOAD" and "STORE" incorporated in instructions	Register to register: "LOAD" and "STORE" are independent instructions
Small code sizes, high cycles per second	Low cycles per second, large code sizes
Transistors used for storing complex instructions	Spends more transistors on memory registers

CISC vs RISC

CISC	RISC
Variable sized instructions	Fixed sized instructions
Can perform REG to REG or REG to MEM or MEM to MEM	Can perform only Register to Register Arithmetic operations
Requires less number of registers	Requires more number of registers
CISC is less pipelined.	RISC is highly pipelined.
Addressing modes are more.	Addressing modes are less.

ALU and Control Unit

- **ALU (Arithmetic Logic Unit):**

- Performs arithmetic (add, subtract) and logic (AND, OR) operations

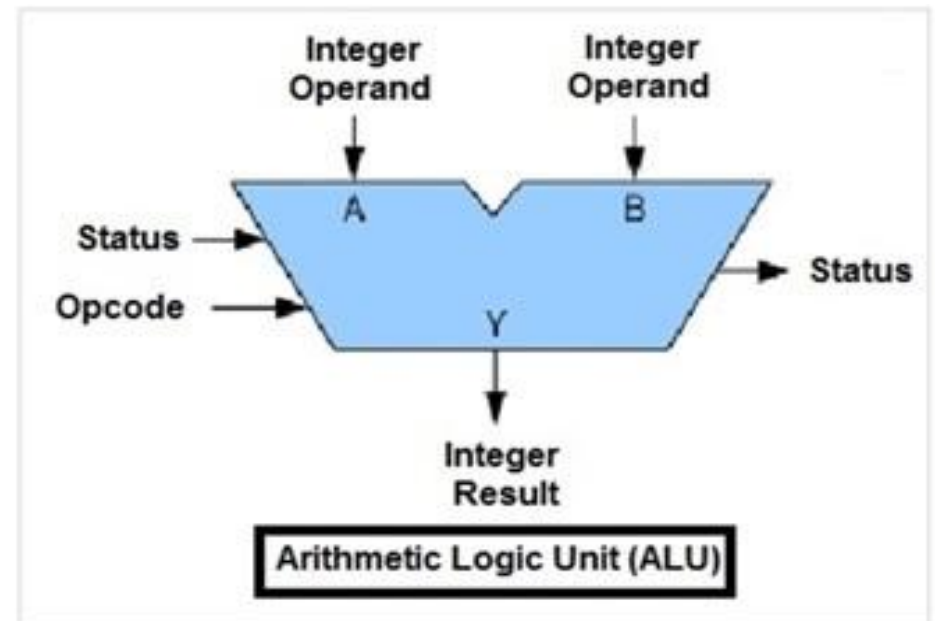
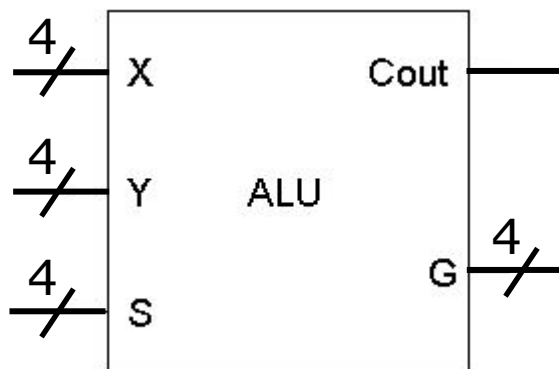
- **Control Unit:**

- Directs operations of the processor
- Interprets instructions and signals components

Arithmetic-logic unit

- ALU conducts arithmetic and logic operations.
- It is a major component of the CPU in a computer system.
- An integer unit (IU) is just an integrated circuit within a CPU or GPU that performs the last calculations in the processor.
- It can execute all arithmetic and logic operations, including Boolean comparisons, such as subtraction, addition, and shifting (XOR, OR, AND, and NOT operations).
- Binary numbers can also perform bitwise and mathematical operations.
- AU (arithmetic unit) and LU (logic unit) are two types of arithmetic logic units.
- The ALU's operands and code instruct it on which operations to perform based on the incoming data.
- When the ALU has finished processing the data, it sends the result to the computer memory.

- This ALU is a good example of hierarchical design.
 - With the 12 inputs, the truth table would have had $2^{12} = 4096$ lines. That's an awful lot of paper.
 - Instead, we were able to use components that we've seen before to construct the entire circuit from a couple of easy-to-understand components.



Control unit

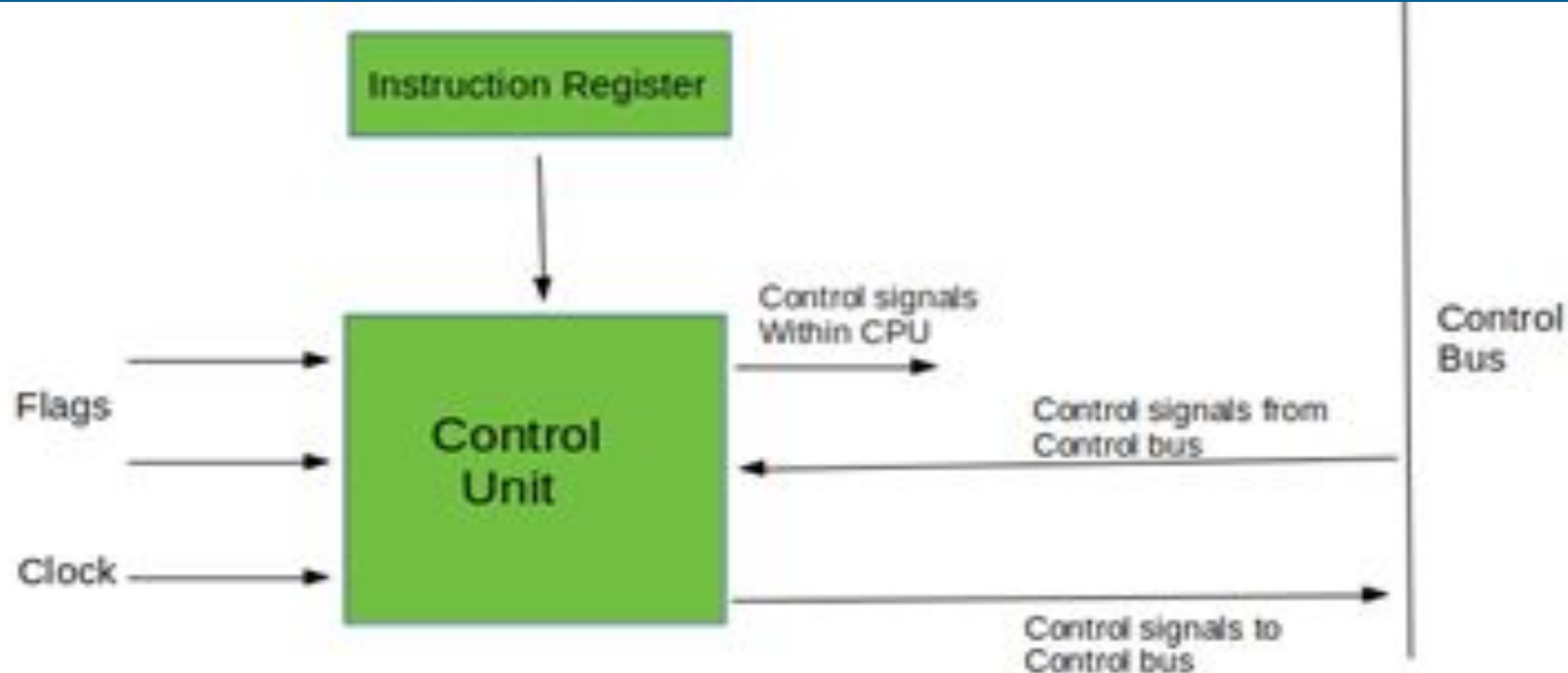
- The control unit (CU) is a component of a CPU that directs the operation of the processor.
- It tells the computer's memory, arithmetic/logic unit(ALU) and input - output devices (I/O) How to respond to a program's instructions ?
- It directs the operation of the other units by providing timing and control signals.
- All computer resources are managed by the CU (Control Unit).
- It directs the flow of data between the CPU and the other devices.
- The control unit communicates / interact with ALU and main memory.
- Control unit generates the control signals to perform the operations.
- It controls the transmission between processor, memory and the various peripherals.
- ²⁹ It also instructs the ALU , which operation should be performed on data.

Design of Control Unit

- Control unit can be designed by two methods :

1. Hardwired Control Unit

2. Micro-Programmed Control Unit.



Block Diagram of the Control Unit

Hardwired Control Unit:

- It is implemented with the help of Physical components which is Hardwired (gates, flip flops, decoders) in the hardware.
- The inputs to control unit are the instruction register, flags, timing signals.
- If design is modified or changed, all the combinational circuits should be modified which is a very difficult task.
- The sequence of the operation carried out by this machine is determined by wiring of the logic elements so, it is known as “Hardwired”.

Micro-programmed Control Unit:

- It is implemented by using Programming approach.
- A sequence of micro operations is carried out by executing a program which consist of micro-instructions.
- In this, any modifications or changes can be done by updating the micro program in the control memory which is done by programmer.
- In this ,Control signals are generated by a program.
- Its speed is slow because of the time it takes to fetch microinstructions from control memory.

Basis	Hardwired Control Unit	Microprogrammed Control Unit
Basic	It is a circuitry approach.	This control unit is implemented by programming
Design	RISC style instructions	CISC style instructions
Modification	Modification is difficult as the control unit is hardwired. Modifying it will require the change in hardware.	Modifications are easy in case of microprogrammed control unit as it will require the in change in the code only.
Instructions	It works well for simple instructions.	It works well for complex instructions also.
Costing	Implementing hardwired structure requires a cost.	Implementing microprograms is not costly.
Control memory	No control memory is required	Control memory is required
Execution Speed	Faster execution	Comparatively slow

Pipelining

Pipeline defines the overlapping of stages for instruction execution.
Pipeline is done to improve execution speed of programs.

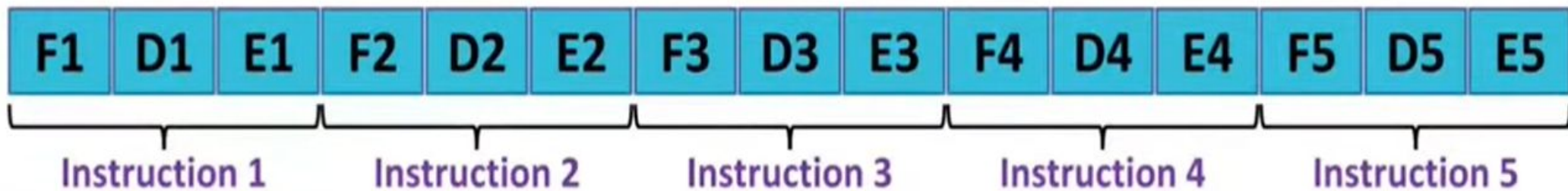
8085
Instruction
Execution



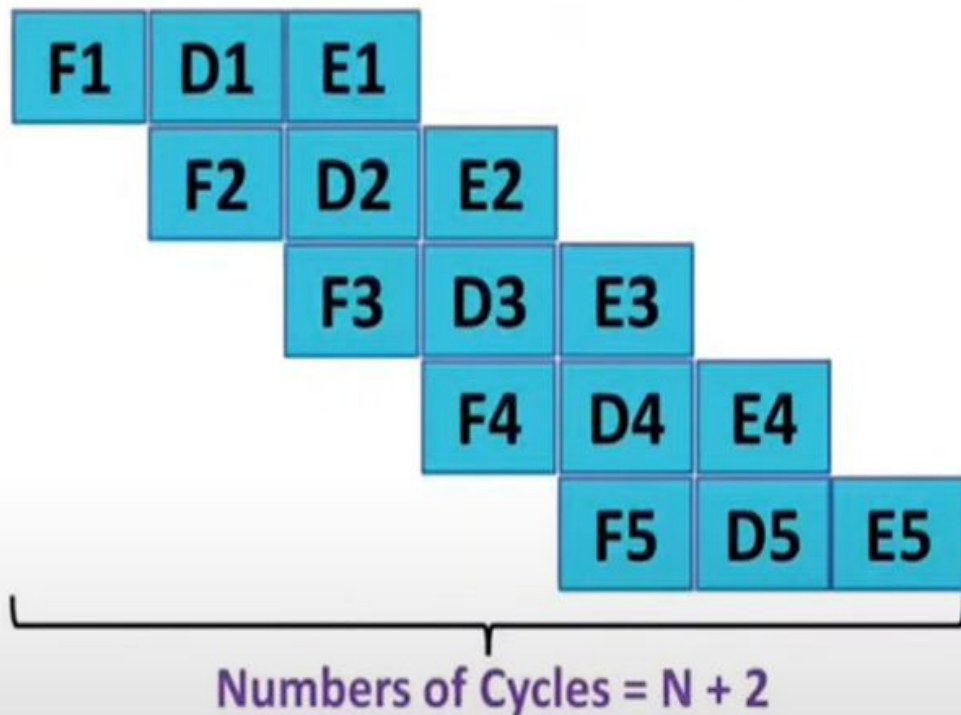
8086
Instruction
Execution



Normal Execution
without Pipelining

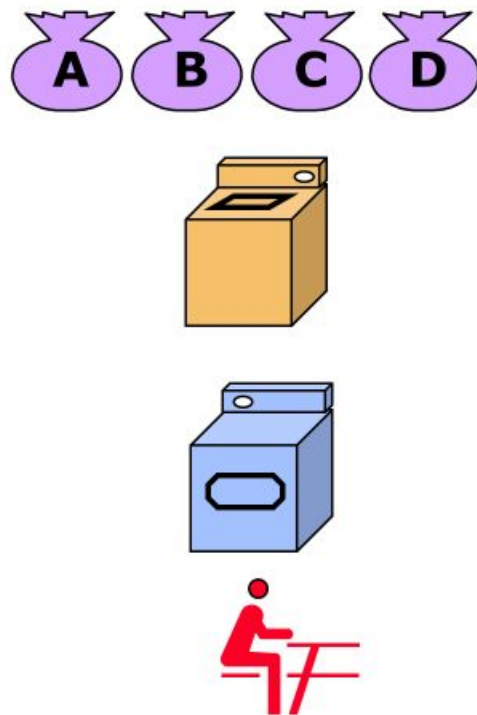


ARM7 with
Pipelining

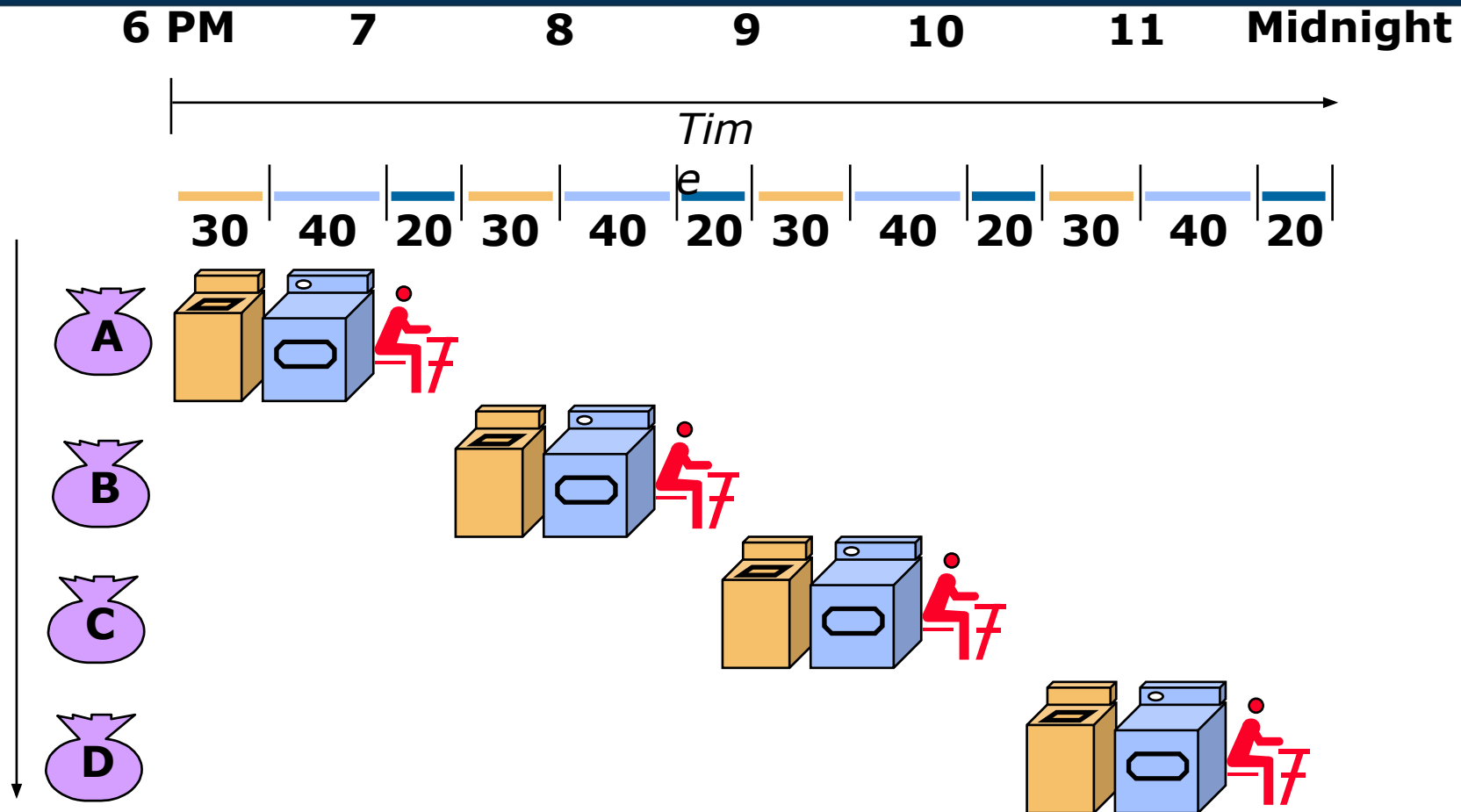


What Is Pipelining?

- ▶ Laundry Example
- ▶ Ann, Brian, Cathy, Dave each have one load of clothes to wash, dry, and fold
- ▶ Washer takes 30 minutes
- ▶ Dryer takes 40 minutes
- ▶ “Folder” takes 20 minutes



What Is Pipelining?

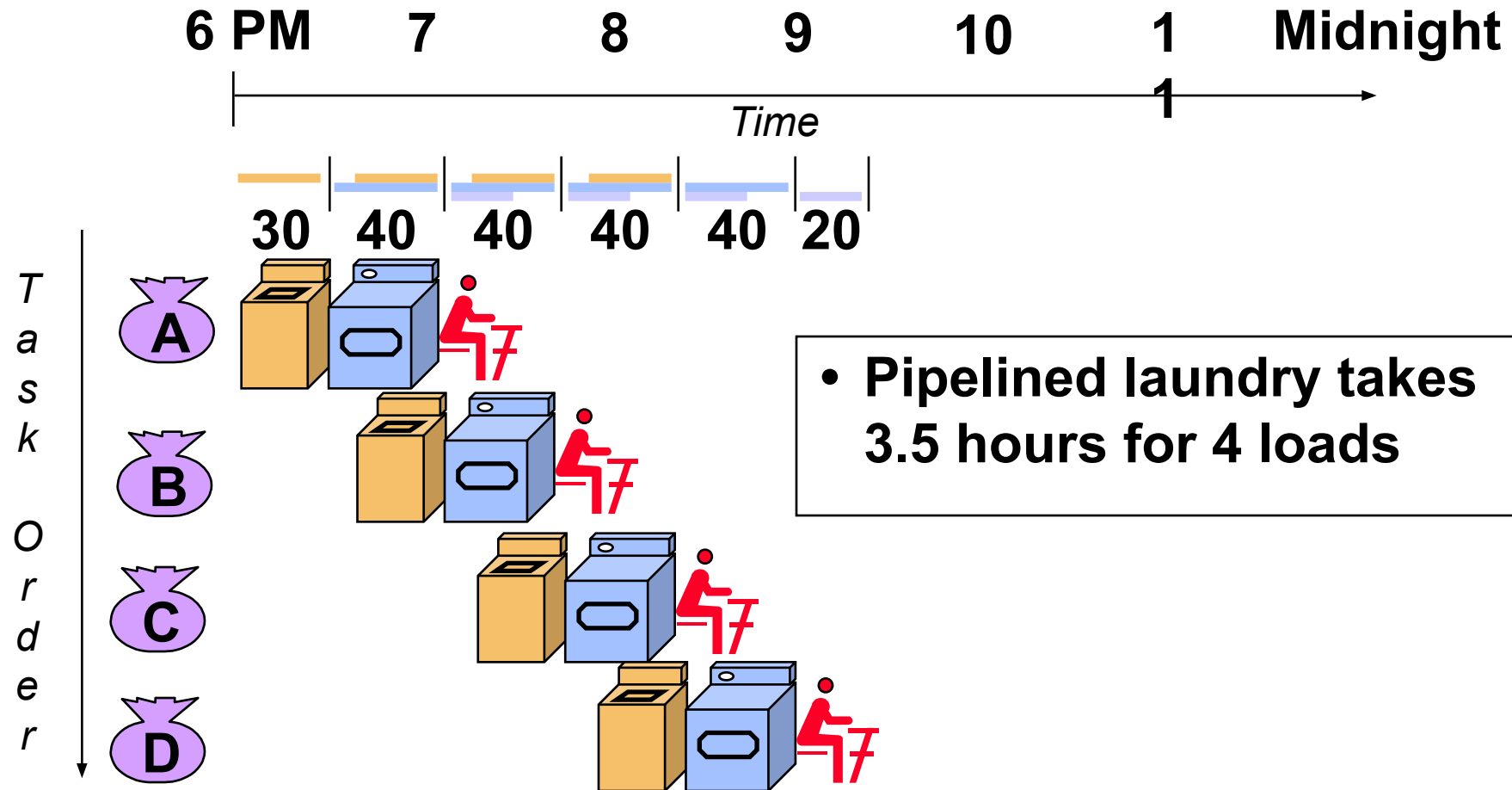


Sequential laundry takes 6 hours for 4 loads

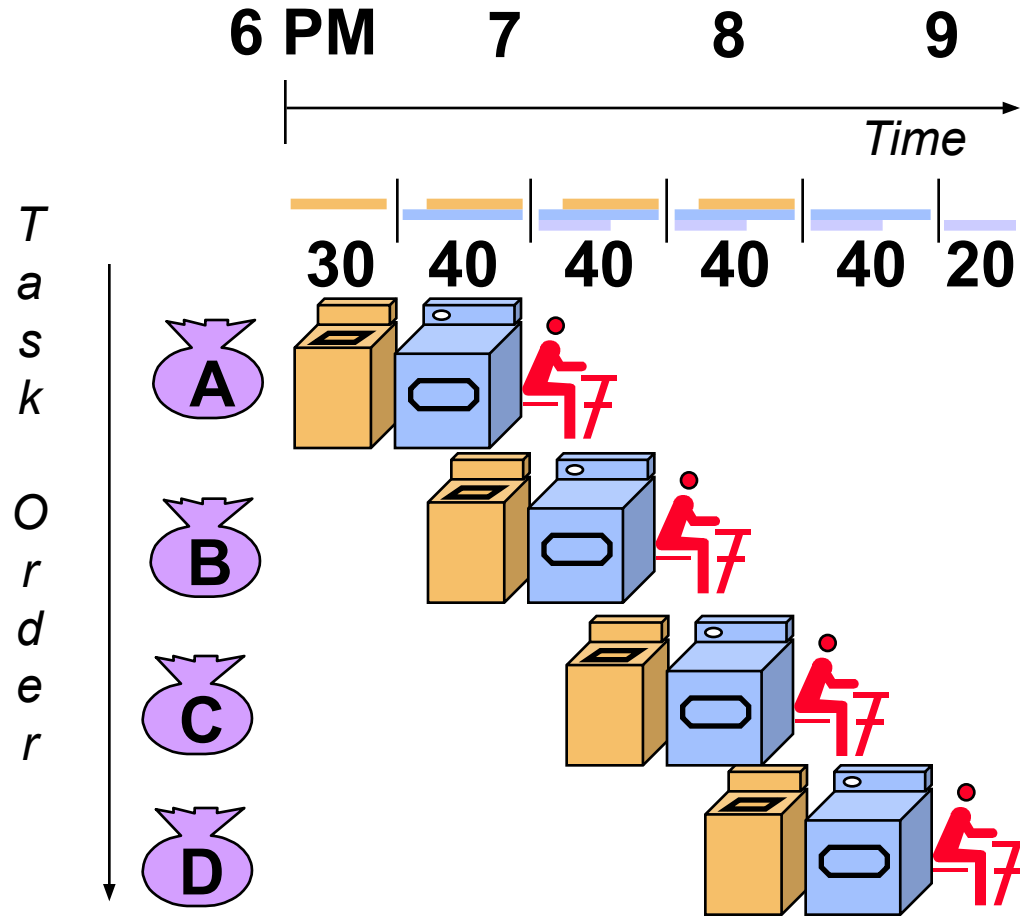
If they learned pipelining, how long would laundry take?

What Is Pipelining

Start work ASAP



What Is Pipelining

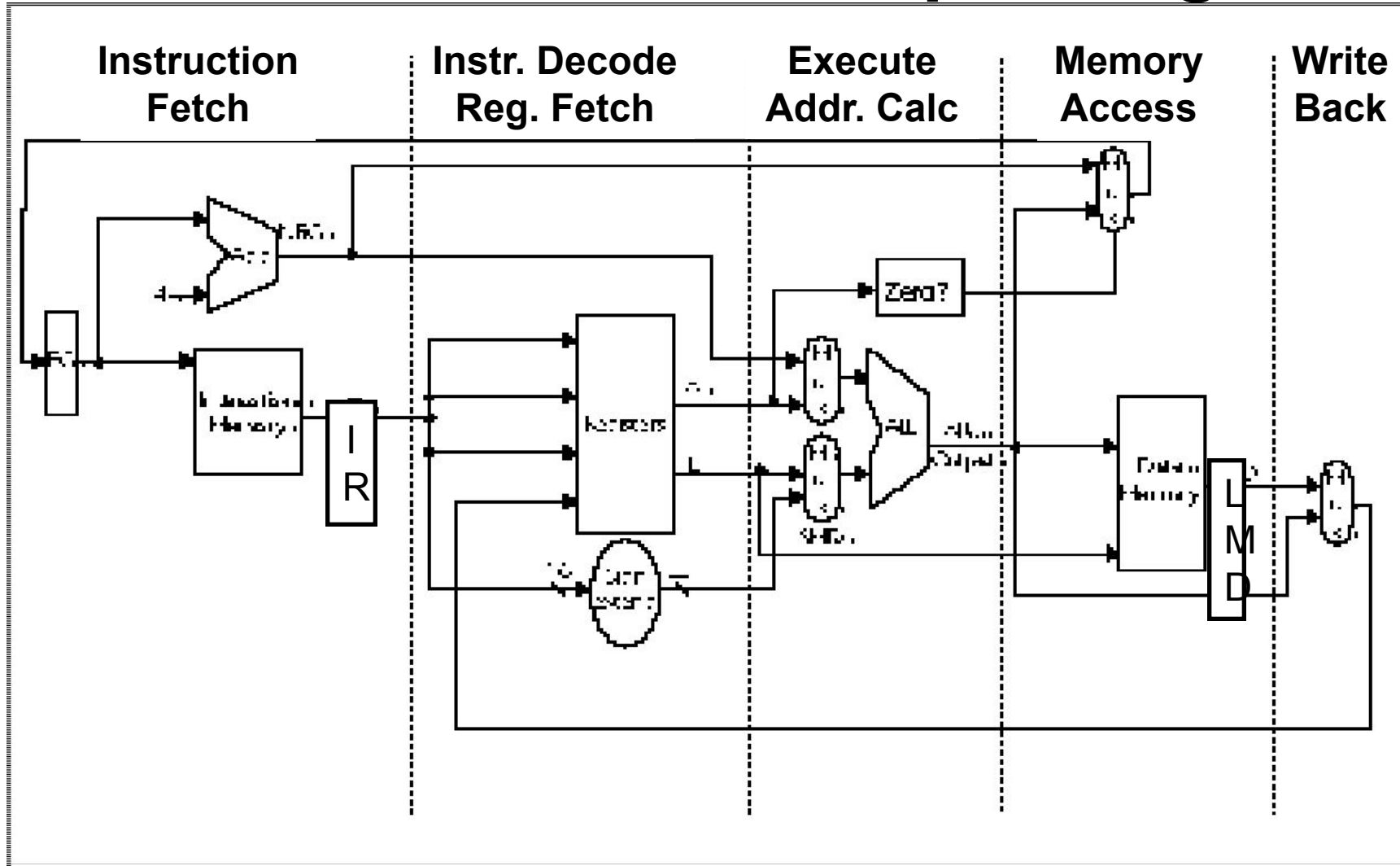


Pipelining Lessons

- Pipelining doesn't help **latency** of single task, it helps **throughput** of entire workload
- Pipeline rate limited by **slowest** pipeline stage
- **Multiple** tasks operating simultaneously
- Potential speedup = **Number pipe stages**
- Unbalanced lengths of pipe stages reduces speedup
- Time to “**fill**” pipeline and time to “**drain**” it reduces speedup

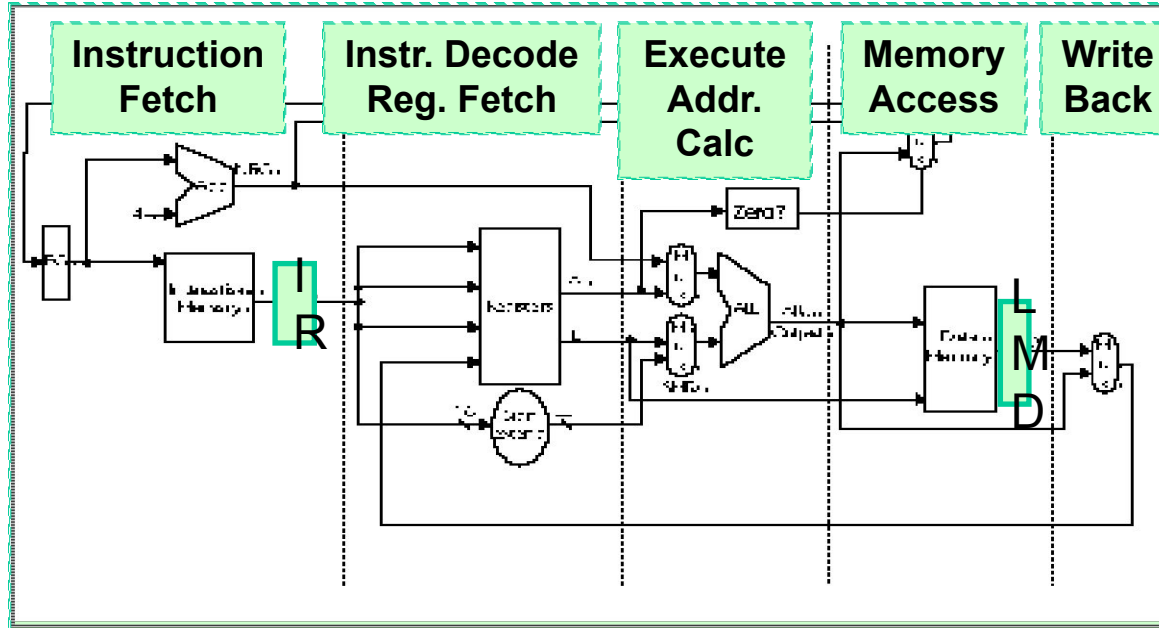
What Is Pipelining

MIPS Without Pipelining



What Is Pipelining

MIPS Functions



Passed To Next Stage

$IR \leftarrow \text{Mem}[PC]$
 $NPC \leftarrow PC + 4$

Instruction Fetch (IF):

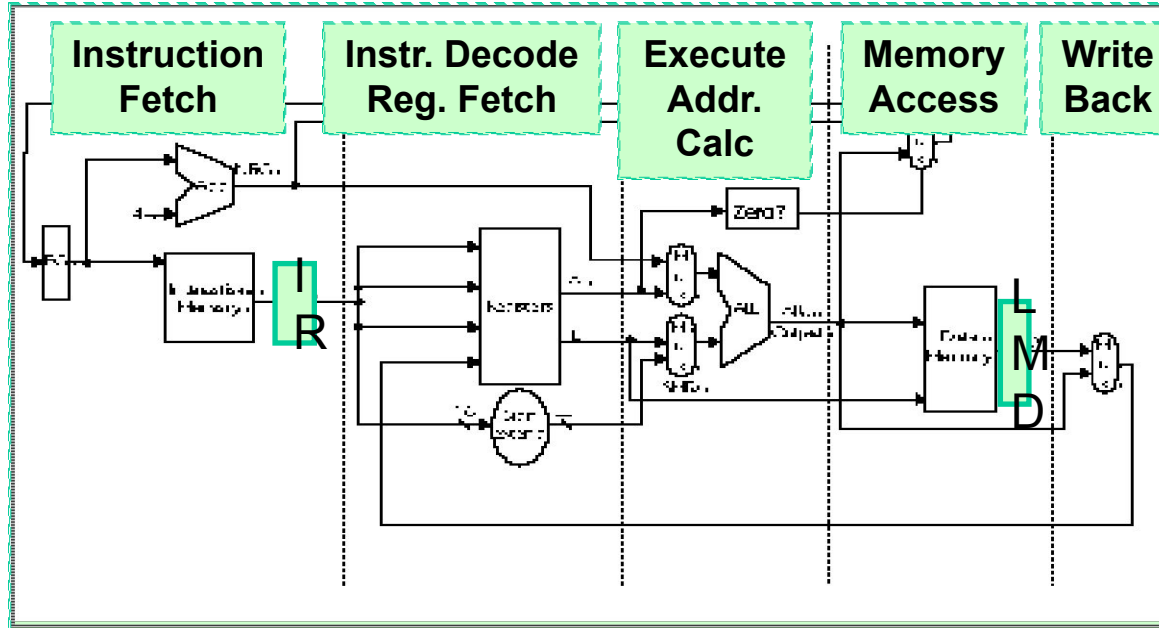
Send out the PC and fetch the instruction from memory into the instruction register (IR); increment the PC by 4 to address the next sequential instruction.

IR holds the instruction that will be used in the next stage.

NPC holds the value of the next PC.

What Is Pipelining

MIPS Functions



Passed To Next Stage

```
A <- Regs[IR6..IR10];  
B <- Regs[IR10..IR15];  
Imm <- ((IR16) ##IR16-31
```

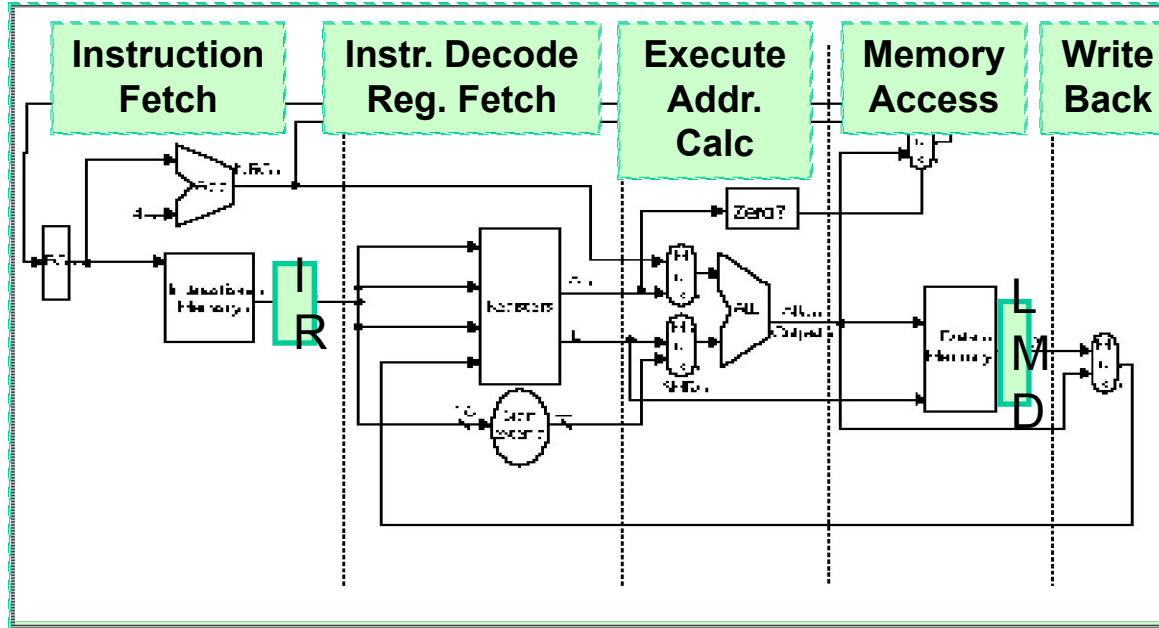
Instruction Decode/Register Fetch Cycle (ID):

Decode the instruction and access the register file to read the registers. The outputs of the general purpose registers are read into two temporary registers (A & B) for use in later clock cycles.

We extend the sign of the lower 16 bits of the Instruction Register.

What Is Pipelining

MIPS Functions



Passed To Next Stage
A <- A func. B
cond = 0;

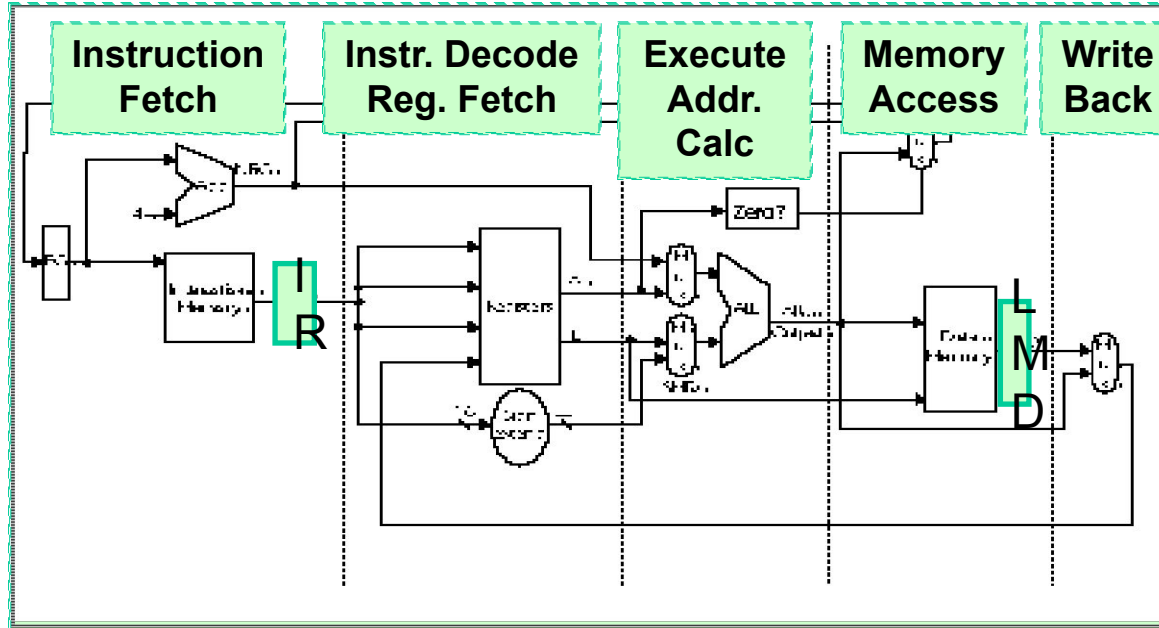
Execute Address Calculation (EX):

We perform an operation (for an ALU) or an address calculation (if it's a load or a Branch).

If an ALU, actually do the operation. If an address calculation, figure out how to obtain the address and stash away the location of that address for the next cycle.

What Is Pipelining

MIPS Functions



Passed To Next Stage
 $A = \text{Mem}[\text{prev. } B]$
or
 $\text{Mem}[\text{prev. } B] = A$

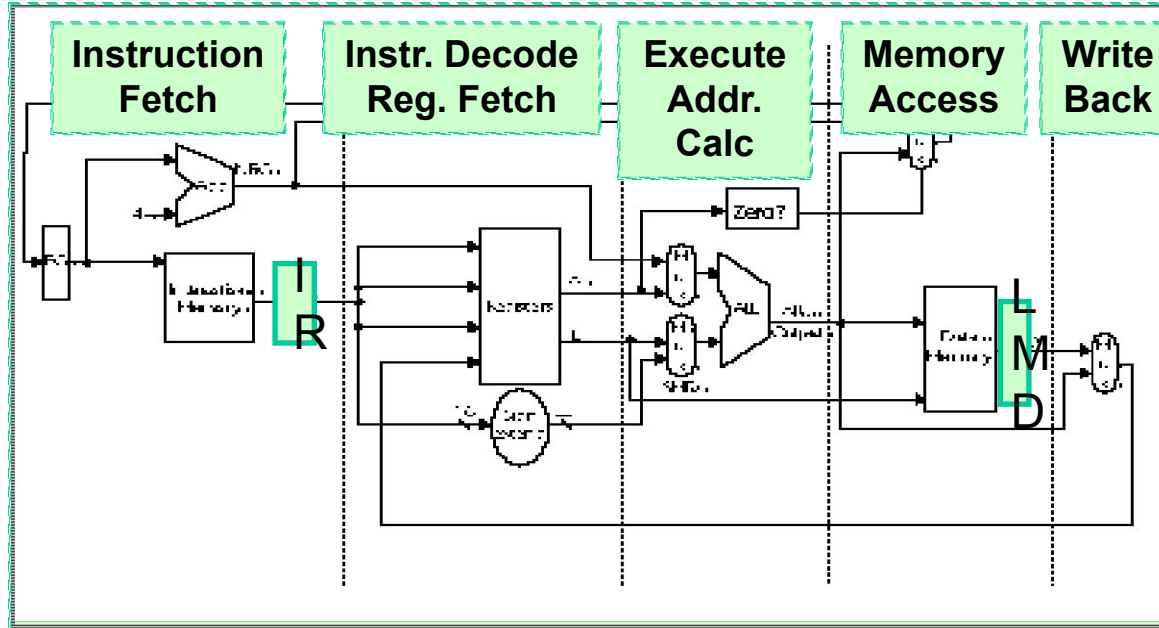
MEMORY ACCESS (MEM):

If this is an ALU, do nothing.

If a load or store, then access memory.

What Is Pipelining

MIPS Functions

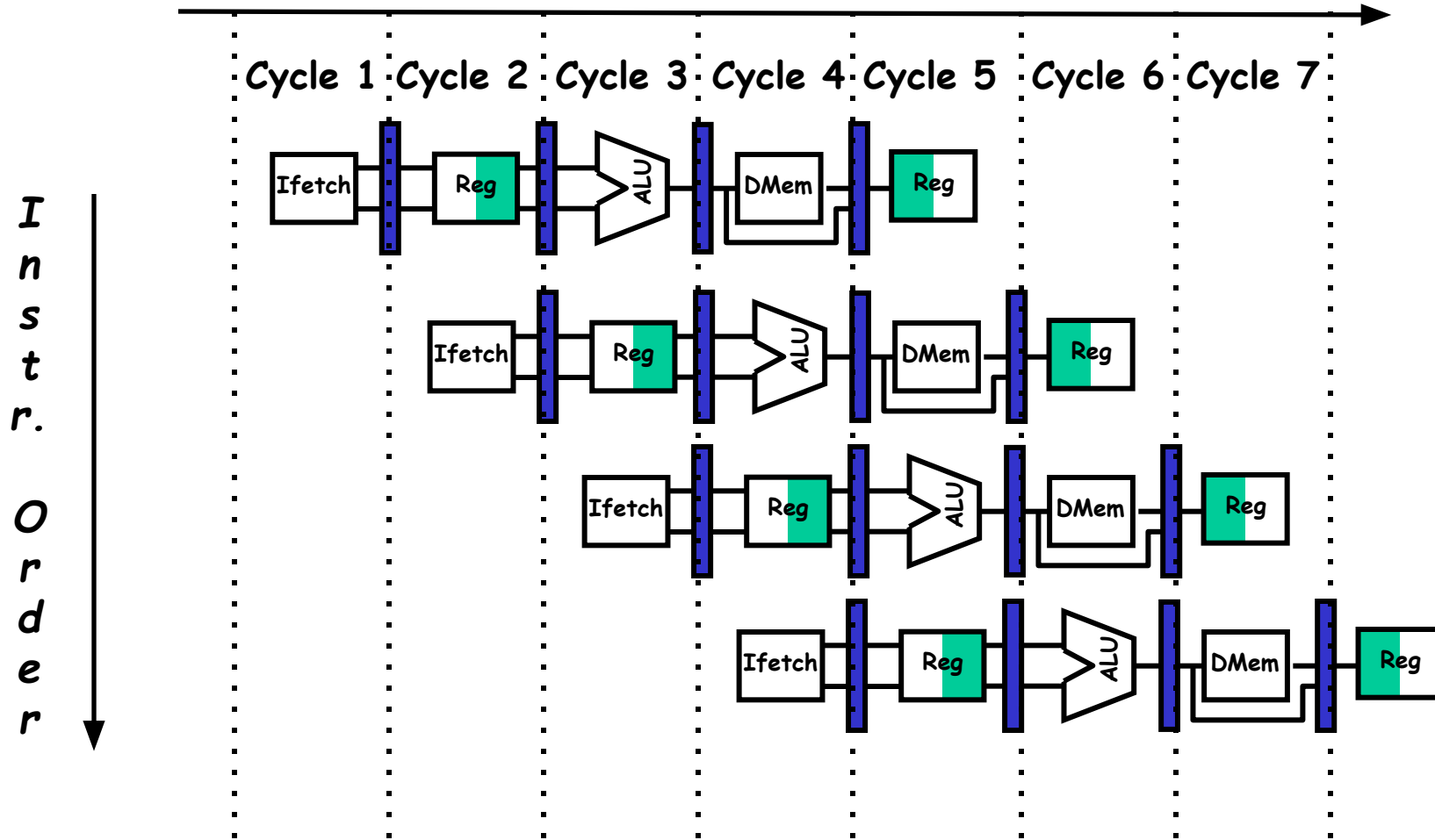


Passed To Next Stage
Regs <- A, B;

WRITE BACK (WB):

Update the registers from either the ALU or from the data loaded.

The Basic Pipeline For MIPS



Pipeline Hurdles

Limits to pipelining: Hazards prevent next instruction from executing during its designated clock cycle

- **Structural hazards**: HW cannot support this combination of instructions (single person to fold and put clothes away)
- **Data hazards**: Instruction depends on result of prior instruction still in the pipeline (missing sock)
- **Control hazards**: Pipelining of branches & other instructions that change the PC
- Common solution is to **stall** the pipeline until the hazard is resolved, inserting one or more “**bubbles**” in the pipeline

Pipeline Hurdles

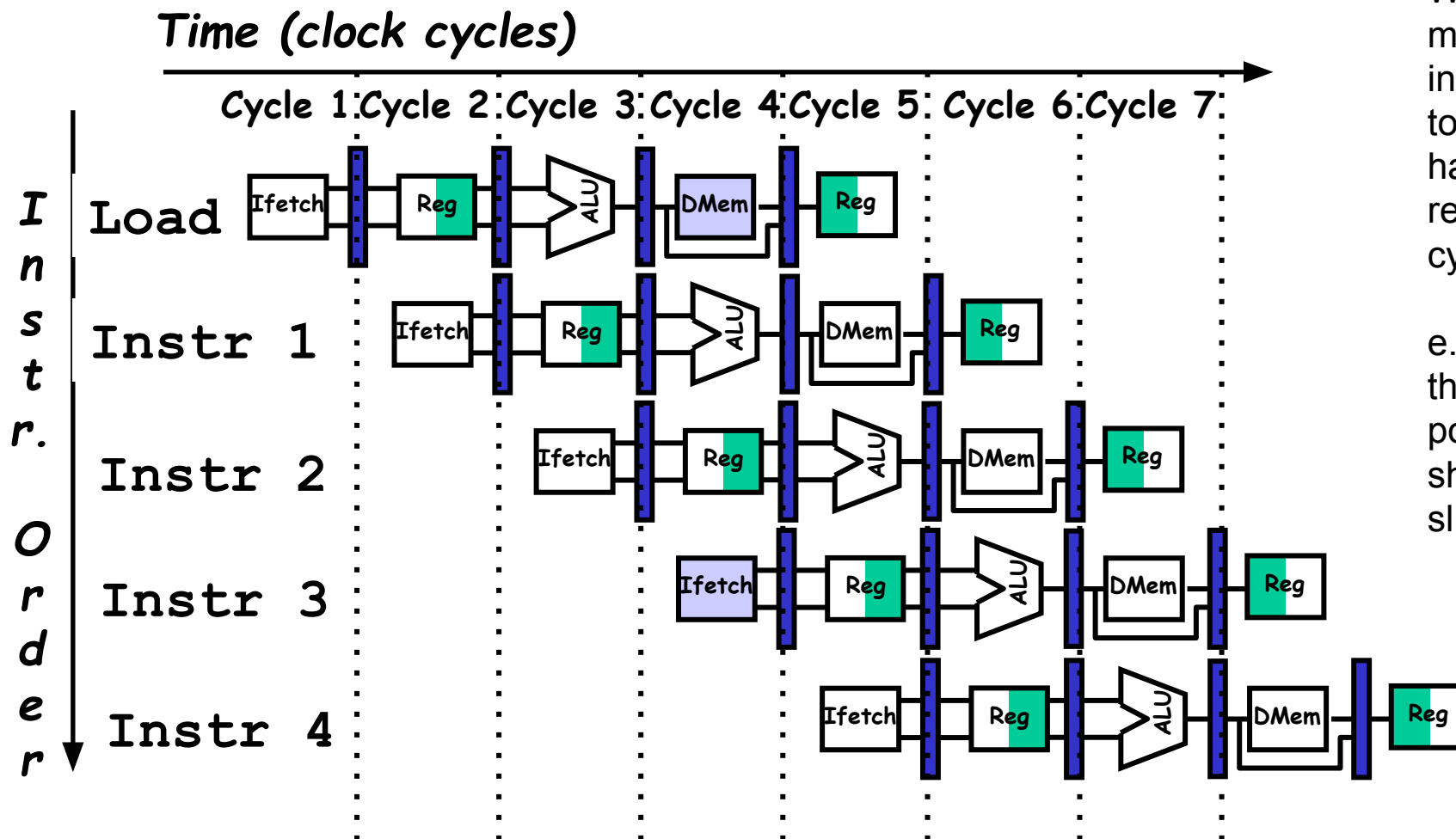
Definition

- conditions that lead to incorrect behavior if not fixed
- Structural hazard
 - two different instructions use same h/w in same cycle
- Data hazard
 - two different instructions use same storage
 - must appear as if the instructions execute in correct order
- Control hazard
 - one instruction affects which instruction is next

Resolution

- Pipeline interlock logic detects hazards and fixes them
- simple solution: stall
- increases CPI, decreases performance
- better solution: partial stall
- some instruction stall, others proceed better to stall early than late

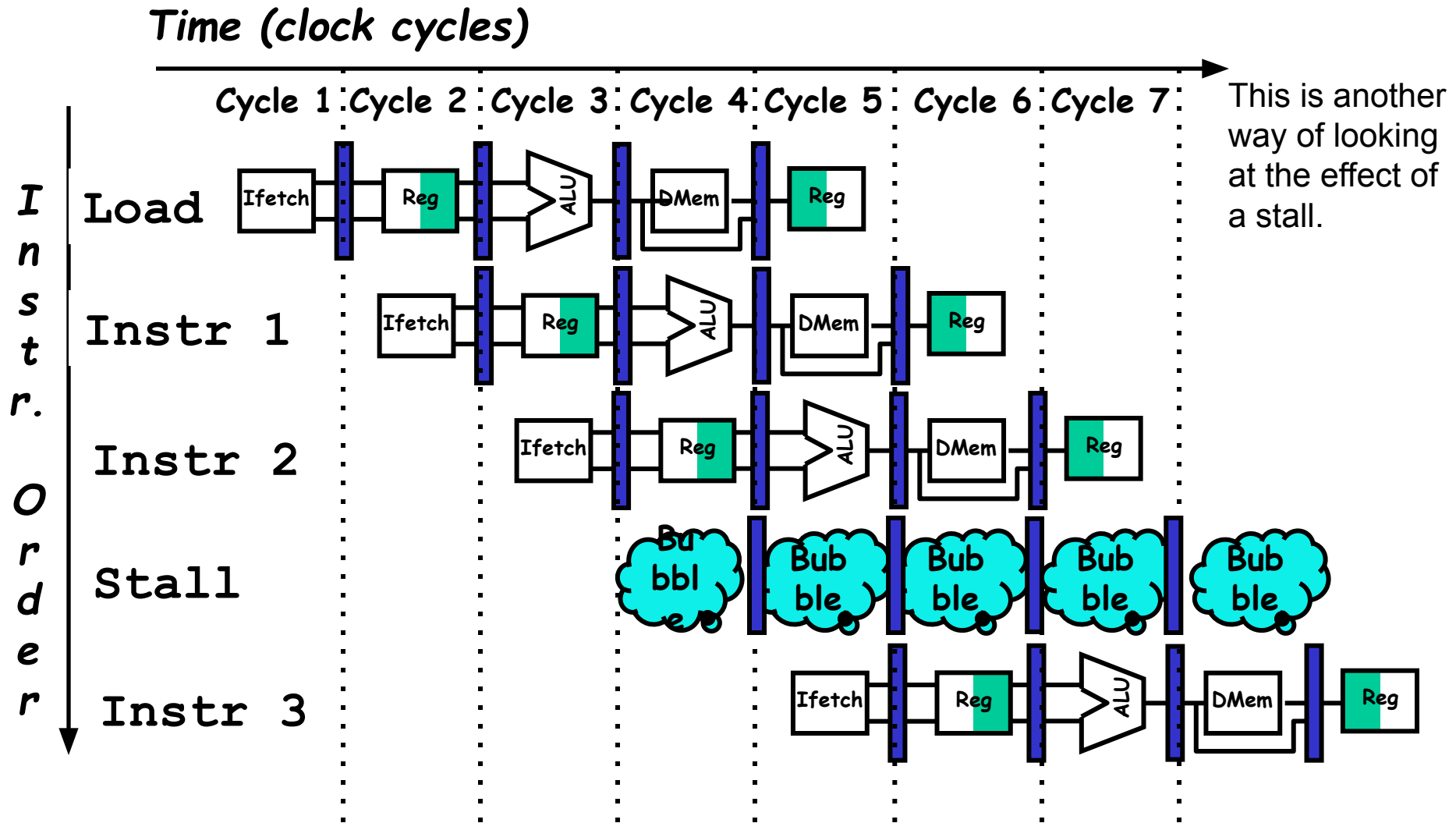
Structural Hazards



When two or more different instructions want to use same hardware resource in same cycle

e.g., MEM uses the same memory port as IF as shown in this slide.

Structural Hazards



Structural Hazards

Instruction	Clock cycle number									
	1	2	3	4	5	6	7	8	9	10
Load instruction	IF	ID	EX	MEM	WB					
Instruction $j+1$		IF	ID	EX	MEM	WB				
Instruction $j+2$			IF	ID	EX	MEM	WB			
Instruction $j+3$				stall	IF	ID	EX	MEM	WB	
Instruction $j+4$						IF	ID	EX	MEM	WB
Instruction $j+5$							IF	ID	EX	MEM
Instruction $j+6$								IF	ID	EX

This is another way to represent the stall.

Structural Hazards

Dealing with Structural Hazards

Structural hazards occur in pipelined processors when hardware resources are not sufficient to handle all active instructions simultaneously. There are several methods to handle such hazards:

1. **Stalling** is the simplest and lowest-cost method to resolve structural hazards. In this approach, the processor delays the execution of certain instructions until the required resource becomes available. While easy to implement, stalling increases the **Cycles Per Instruction (CPI)**, which negatively affects performance. Therefore, it is typically used only in rare situations where hazards are infrequent.
2. **Pipelining the hardware resource** is another approach, where resources such as memory (especially those requiring multiple cycles) are broken into pipeline stages. This method can offer good performance by allowing overlapping use of the resource. However, it can also introduce complexity in the design, particularly when dealing with components like RAM.
3. **Replicating the resource** involves duplicating the hardware unit so that multiple instructions can use it simultaneously without conflict. This method achieves high performance but at the cost of increased hardware expense. Additionally, it may introduce interconnection delays. This approach is most useful when the resource is inexpensive or can be easily divided into multiple independent units.

Structural Hazards

Structural hazards are reduced with these rules:

- Each instruction uses a resource at most once
- Always use the resource in the same pipeline stage
- Use the resource for one cycle only

Many RISC ISA's designed with this in mind

Sometimes very complex to do this.

For example, memory of necessity is used in the IF and MEM stages.

Some common Structural Hazards:

- Memory
- Registers
- ALU

Data Hazards

These occur when at any time, there are instructions active that need to access the same data (memory or register) locations.

Where there's real trouble is when we have:

instruction A

instruction B

and B manipulates (reads or writes) data before A does. This violates the order of the instructions, since the architecture implies that A completes entirely before B is executed.

Data Hazards

Execution Order is:

Instr_i
Instr_j

Read After Write (RAW)

Instr_j tries to read operand before Instr_i writes it

 I: add r1, r2, r3
J: sub r4, r1, r3

- Caused by a “**Dependence**”.
- This hazard results from an actual need for communication.

Data Hazards

Execution Order is:

Instr_i
Instr_j

Write After Read (WAR)

Instr_j tries to write operand before Instr_i reads it

- Gets wrong operand

```
    I:  sub  r4, r1, r3
    J:  add  r1, r2, r3
    K:  mul  r6, r1, r7
```

- Called an “anti-dependence” by compiler writers.
This results from reuse of the name “r1”.

- **Can't happen in MIPS 5 stage pipeline because:**
 - All instructions take 5 stages, and
 - Reads are always in stage 2, and
 - Writes are always in stage 5

Data Hazards

Execution Order is:

Instr_i
Instr_j

Write After Write (WAW)

Instr_j tries to write operand before **Instr_i** writes it

- Leaves wrong result (Instr_i not Instr_j)

 I: sub r1, r4, r3
J: add r1, r2, r3
K: mul r6, r1, r7

- Called an “**output dependence**” by compiler writers
This also results from the reuse of name “**r1**”.
- Can’t happen in MIPS 5 stage pipeline because:
 - All instructions take 5 stages, and
 - Writes are always in stage 5

Data Hazards

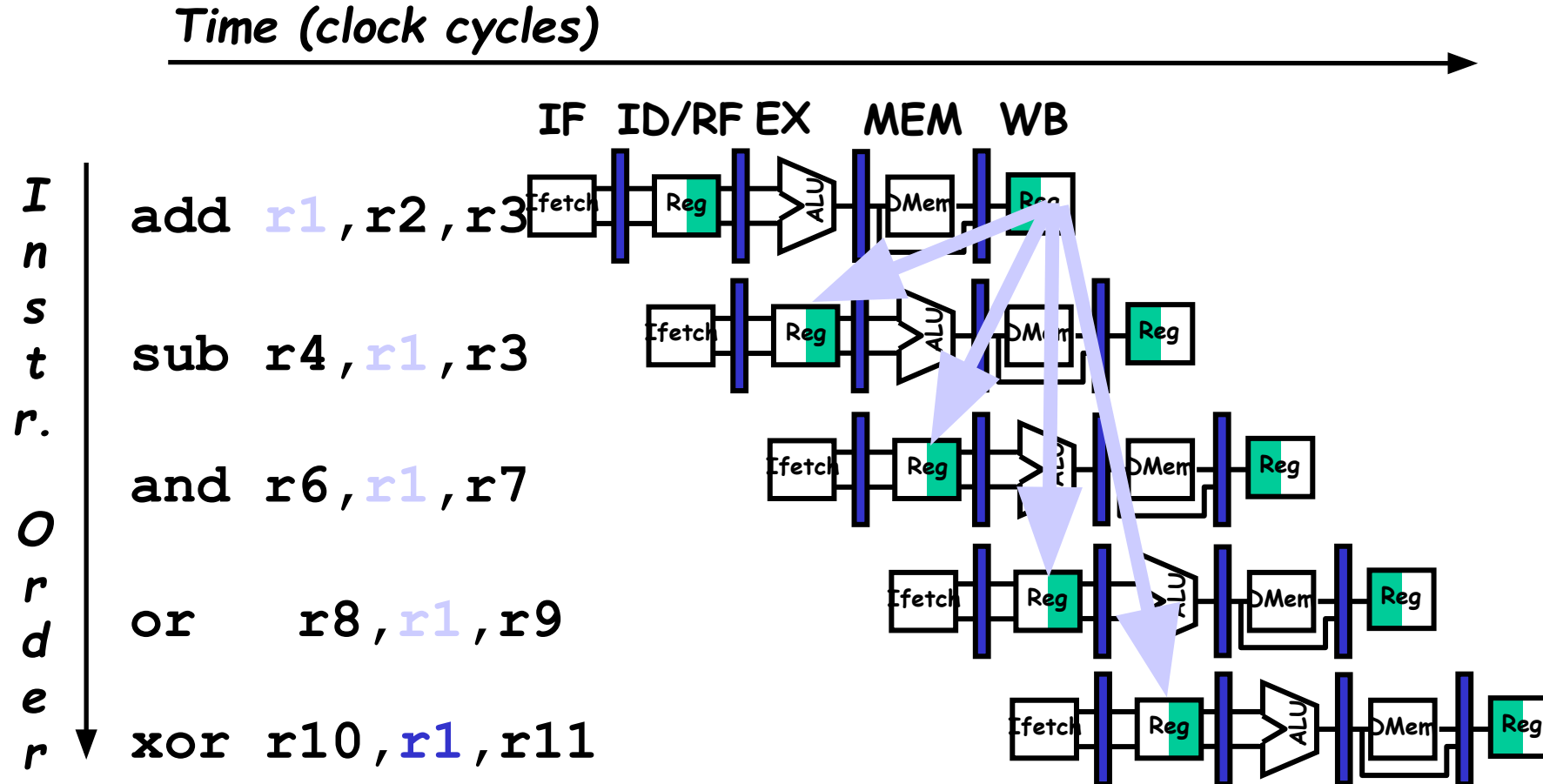
- **Simple Solution to RAW (Read After Write) Hazards**

A basic way to handle RAW hazards is to have the hardware detect them and insert a stall in the pipeline. This method assumes that a register is written in one cycle and then read in the next. It is a low-cost and simple solution to implement. However, it reduces the **Instructions Per Cycle (IPC)**, which affects overall performance. Therefore, efforts are usually made to minimize these stalls as much as possible.

- **Minimizing RAW Stalls**

One effective technique to reduce RAW stalls is **bypassing**, also known as **forwarding** or **short-circuiting**. This method allows data to be used before it is officially written back to the register file. By doing so, it helps avoid unnecessary pipeline stalls. Although forwarding adds complexity to the hardware, it is crucial for efficiently handling common RAW hazards and significantly improves performance.

Data Hazards

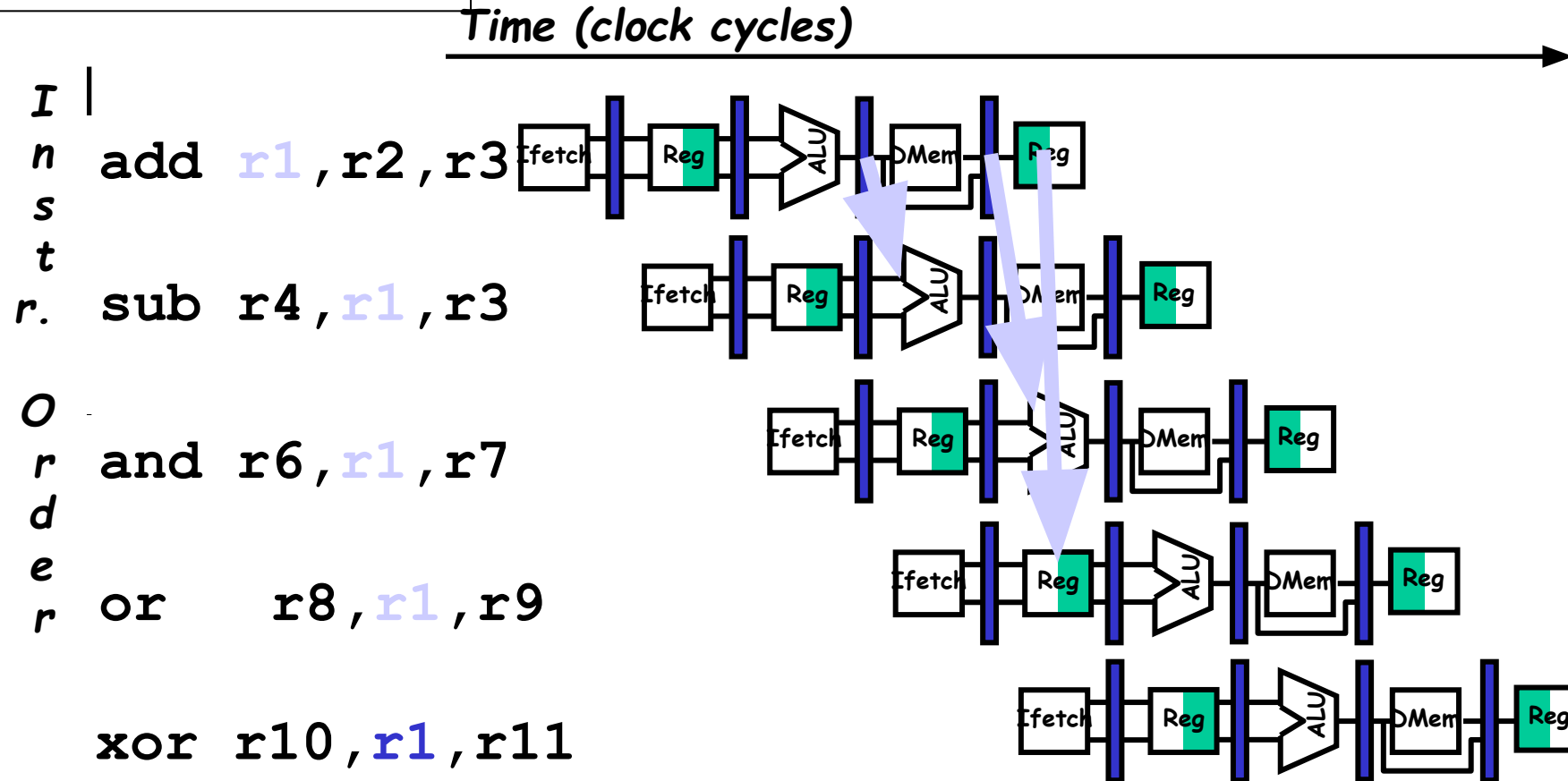


The use of the result of the ADD instruction in the next three instructions causes a hazard, since the register is not written until after those instructions read it.

Data Hazards

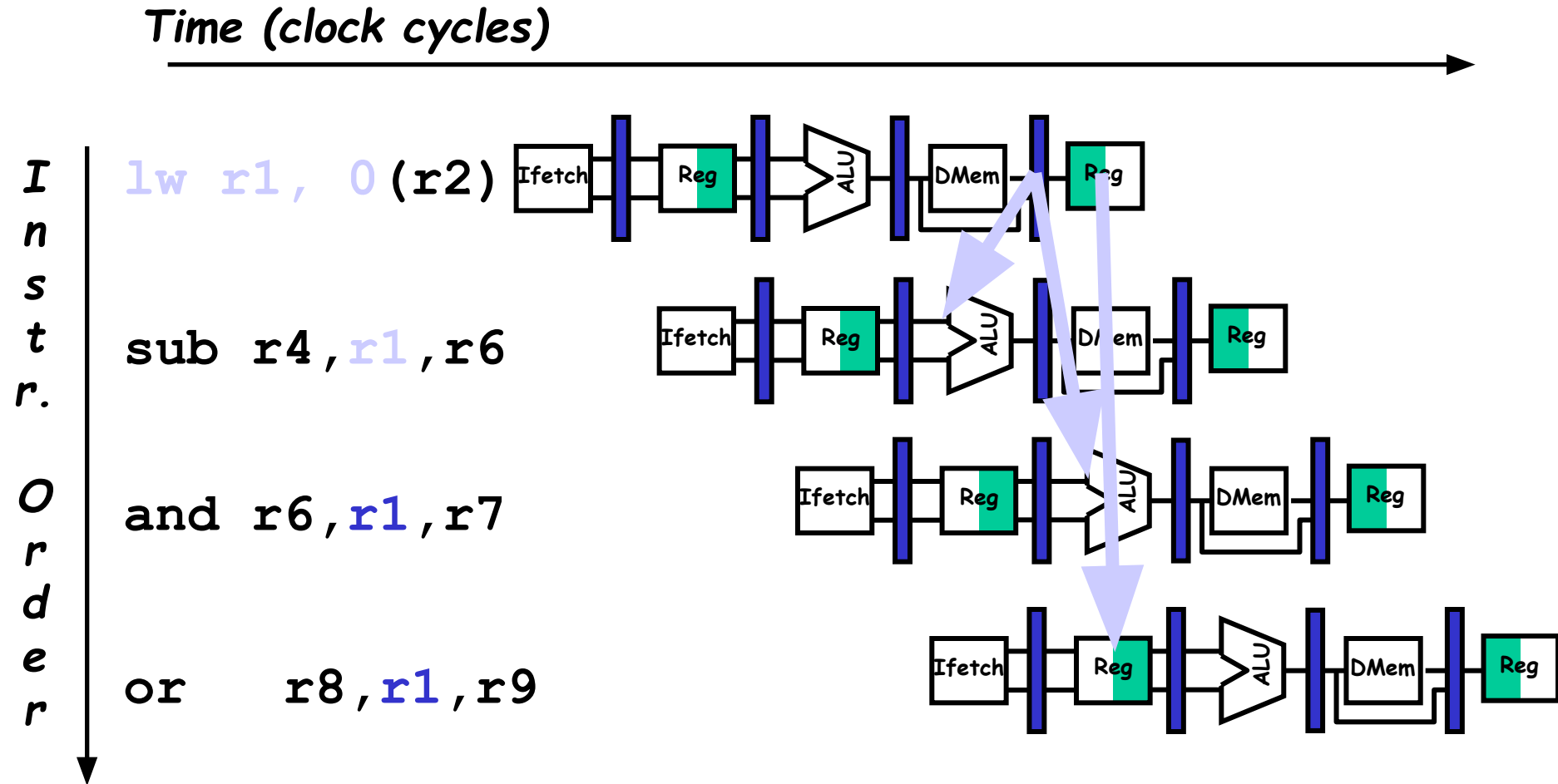
Forwarding To Avoid
Data Hazard

Forwarding is the concept of making data available to the input of the ALU for subsequent instructions, even though the generating instruction hasn't gotten to WB in order to write the memory or registers.



Data Hazards

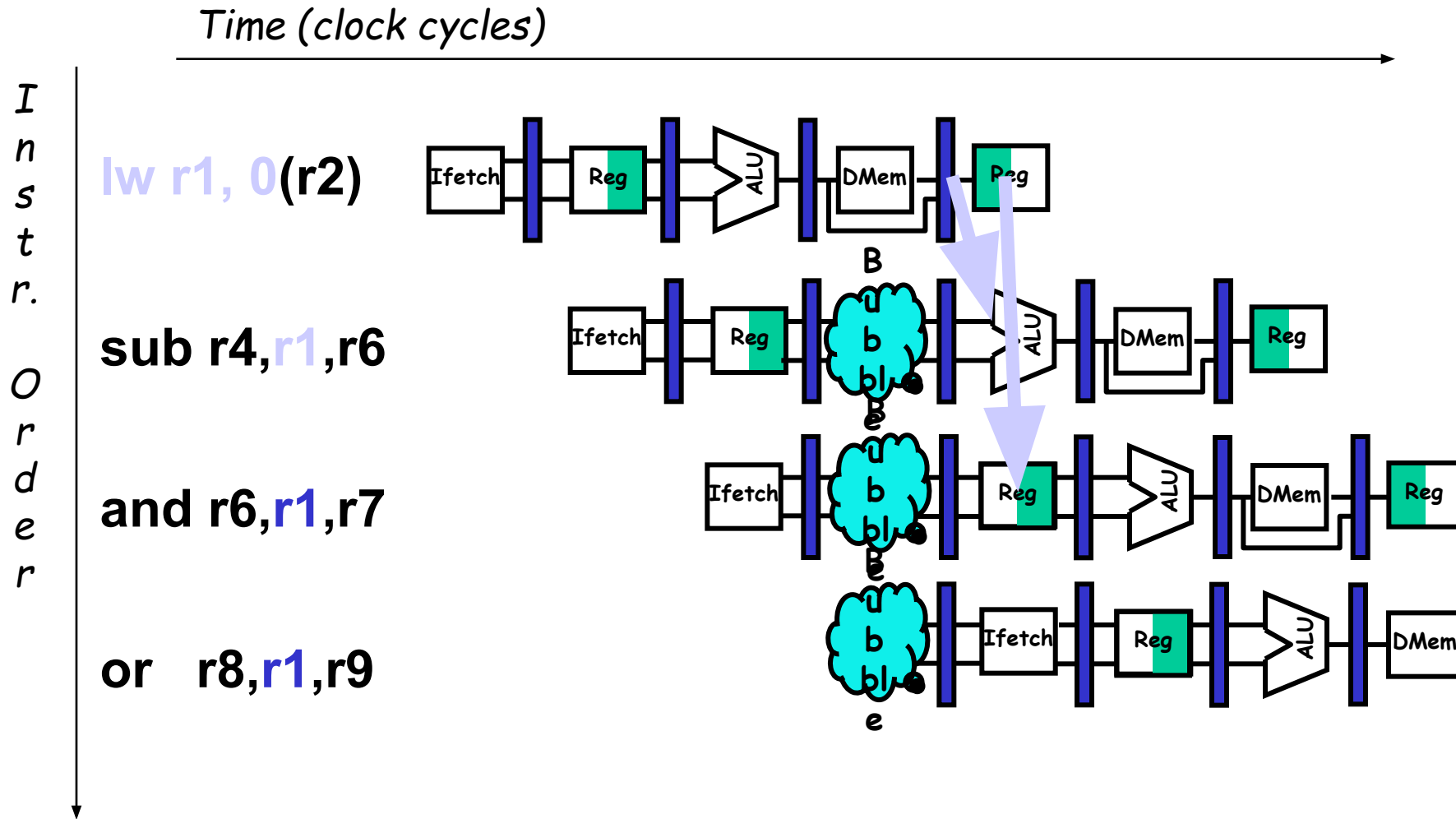
The data isn't loaded until after the MEM stage.



There are some instances where hazards occur, even with forwarding.

Data Hazards

The stall is necessary as shown here.



There are some instances where hazards occur, even with forwarding.

Data Hazards

This is another representation of the stall.

LW	R1, 0(R2)	IF	ID	EX	MEM	WB			
SUB	R4, R1, R5		IF	ID	EX	MEM	WB		
AND	R6, R1, R7			IF	ID	EX	MEM	WB	
OR	R8, R1, R9				IF	ID	EX	MEM	WB

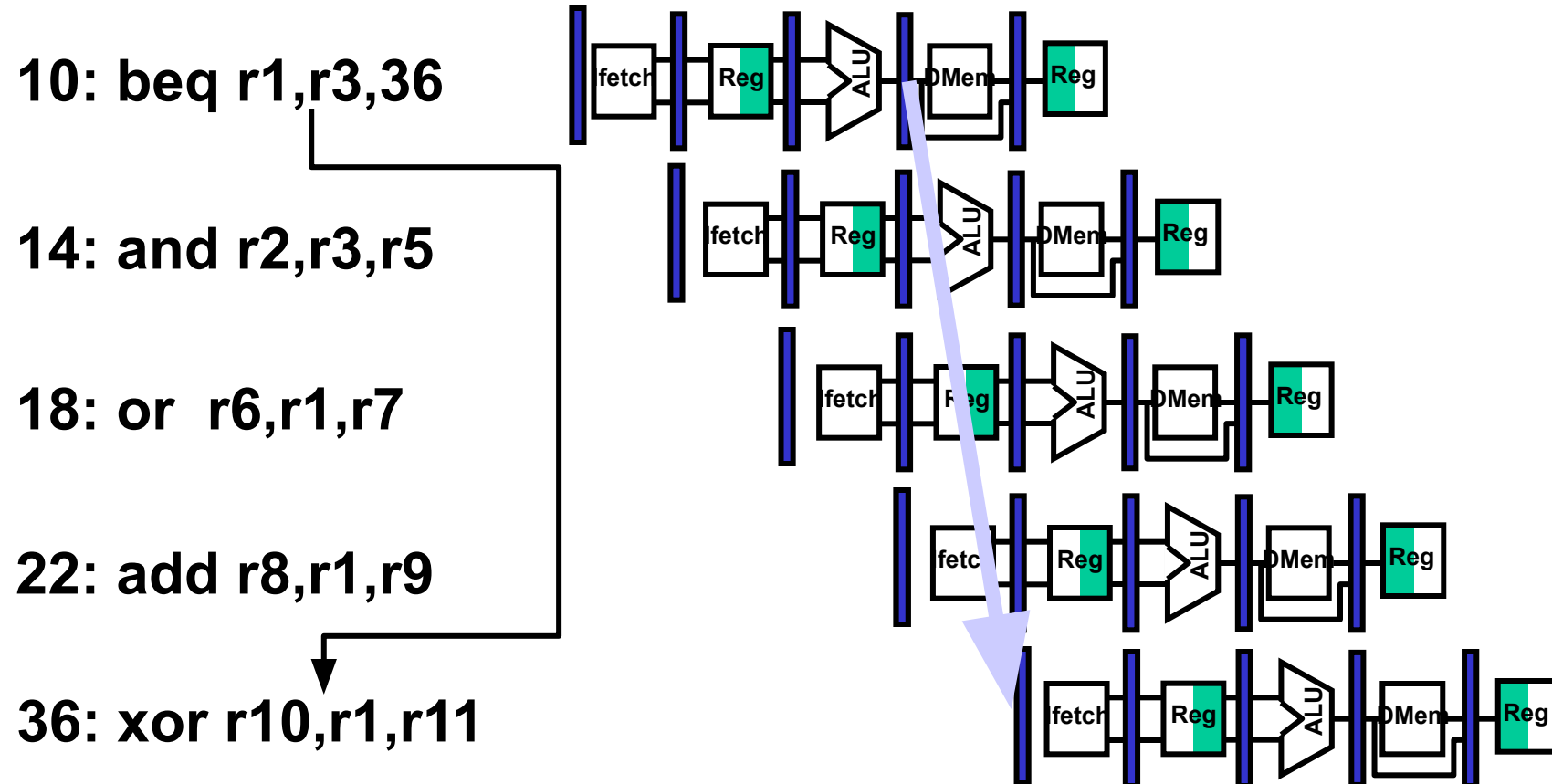
LW	R1, 0(R2)	IF	ID	EX	MEM	WB				
SUB	R4, R1, R5		IF	ID	stall	EX	MEM	WB		
AND	R6, R1, R7			IF	stall	ID	EX	MEM	WB	
OR	R8, R1, R9				stall	IF	ID	EX	MEM	WB

Control Hazards

A control hazard is when we need to find the destination of a branch, and can't fetch any new instructions until we know that destination.

Control Hazards

Control Hazard on Branches Three Stage Stall



- Branch hazards are caused by branch instructions and are known as control hazards.
- The flow of program/instruction execution is controlled by branch instructions.
- In higher-level languages, conditional statements are used for repetitive loops or condition testing (correlate with while, for, if, case statements).
- These are converted into one of the BRANCH instruction variations.
- To understand the programme flow, you must know the value of the condition being tested. It's a difficult situation.
- As a result, when the decision to execute one instruction is reliant on the result of another instruction, such as a conditional branch, which examines the condition's consequent value, a conditional hazard develops.
- The Program Counter (PC) is loaded with the appropriate place for the branch and jump instructions, which determines the programme flow. The next instruction that is to be fetched and executed by the given CPU is stored in the PC.

Conditional Hazards Solutions

1. Stall

Stall the given pipeline as soon as any branch instructions are decoded. Stalling reduces throughput as it always does. According to statistics, at least 30% of the instructions in a program are BRANCH. With Stalling, the pipeline is effectively operating at 50% capacity.

2. Prediction

Consider a for or a while loop that is repeated 100 times. We know the programme would run 100 times without the given branch condition being met. The program only exits the loop for the 101st time. As a result, it's better to let the pipeline run its course and then flush/undo when the branch condition is met. This has less of an impact on the pipeline's throttle and stalling.

3. Dynamic Branch Prediction

With the help of Branch Table Buffer, a historical record is kept (BTB). The BTB is a type of cache that contains a series of entries containing the branch instruction's PC address and the effective branch address. This is done for each branch instruction that is encountered. When a conditional branch instruction is encountered, the BTB is queried for the matching branch instruction address. If the target branch address is hit, the next instruction is fetched from the associated target branch address. Dynamic branch prediction is the term for this.

Branch Instruction Address	Target Branch Address Taken

4. Reordering Instructions

Delayed branching entails reordering the instructions to move the branch instruction later in the sequence, allowing safe and beneficial instructions that are unaffected by the result of a branch to be brought in earlier in the sequence, delaying the fetch of the branch instruction. If such instructions are not available, NOP is used. The Compiler is used to implement this delayed branch.

Memory Organization & Types

- Memory organization is essential for efficient data processing and storage.
- The memory hierarchy ensures quick access to data by the CPU, while larger, slower storage devices hold data for the long term.
- Effective memory management ensures the system operates efficiently, providing programs with the memory they need and preventing unnecessary delays in processing.
- A memory unit is the collection of storage units or devices together. The memory unit stores the binary information in the form of bits. Generally, memory/storage is classified into 2 categories:
 1. **Volatile Memory:** This loses its data, when power is switched off.
 2. **Non-Volatile Memory:** This is a permanent storage and does not lose any data when power is switched off.

Memory Organization & Types

Von Neumann Architecture:

The Von Neumann architecture is one of the most widely used memory organizations in computer systems today. It is named after the computer scientist **John von Neumann**, who first proposed this architecture in the **1940s**. In the Von Neumann architecture, the computer's **memory is divided into two parts: the data memory and the program memory**. Both data and instructions are stored in the same memory and accessed through the same bus. This makes it easy for the computer to execute instructions, but it can also lead to performance bottlenecks when the CPU needs to access both data and instructions at the same time.

Harvard Architecture:

The Harvard architecture is a memory organization that **separates the data memory and the program memory into two separate physical memories, which are accessed by separate buses**. This allows the CPU to access both data and instructions simultaneously, which can improve performance. However, it also requires more hardware, which can increase the cost and complexity of the system.

Memory Organization & Types

Cache Memory:

Cache memory is a type of memory organization that is used to improve the performance of a computer system. Cache memory is a small amount of **high-speed memory that is located close to the CPU**. When the CPU needs to access data, it first checks the cache memory to see if the data is already there. If the data is in the cache memory, it can be accessed quickly, which can significantly reduce the time it takes for the CPU to access the data.

Virtual Memory:

Virtual memory is a memory organization that **allows a computer system to use more memory than is physically available**. This is accomplished by temporarily transferring data from the computer's main memory to disk storage. When the data is needed again, it is transferred back to the main memory. Virtual memory allows programs to run even if the computer does not have enough physical memory to accommodate all of the data and instructions that the program requires.

The Memory Hierarchy

Increase in

- Storing capacity
- Transfer Time
- Access time



Registers

Cache
Memory

Main Memory

Magnetic Disks and
Optical Disks

Magnetic Tapes

Increase in

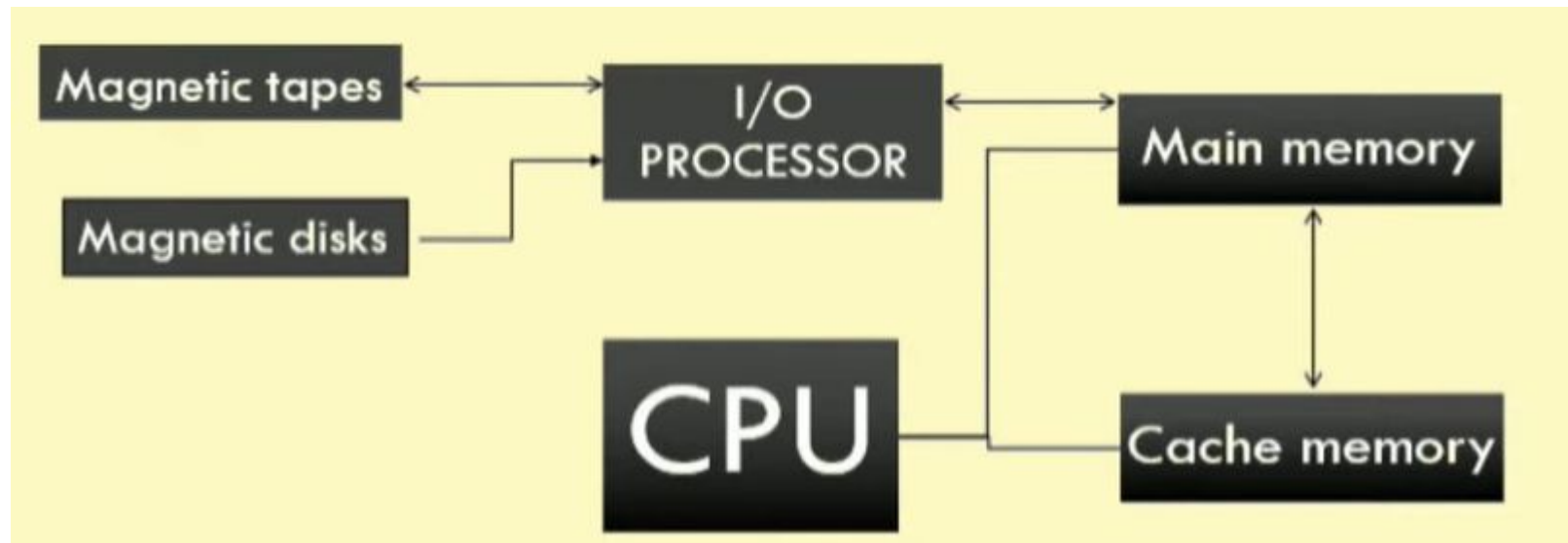
- Cost per Bit
 - Speed
 - Transfer Rate
- Decrease in size.



The Memory Hierarchy

The memory unit that communicates directly with CPU is called **Main Memory**.

Devices that provides backup storage are called **Auxiliary Memory**.



The Memory Hierarchy

Auxiliary Memory (Non-Volatile)

Devices that provide secondary or backup storage are called auxiliary memory. For example, Magnetic disks and tapes are commonly used auxiliary devices. It is not directly accessible to the CPU, is accessed using the Input/Output channels.

- **Hard Disk Drive (HDD):** A permanent storage device that holds large amounts of data even when the computer is turned off. It is slower than RAM but offers much more capacity.
- **Solid-State Drive (SSD):** A faster alternative to HDDs with no moving parts. SSDs provide faster read/write speeds compared to HDDs.
- **Optical Discs and USB Flash Drives:** Optical discs and USB flash drives are other forms of secondary memory used for storage, though they are less common in modern high-speed systems.

Main Memory (Volatile)

The memory unit that communicates directly within the CPU, Cache memory is called main memory. It is fast memory used to store data during computer operations. Main memory is made up of **RAM** and **ROM**, majority part consists of RAM.

The Memory Hierarchy

RAM Random Access Memory

- **DRAM:** Dynamic RAM, is made of capacitors and transistors. It is slower and cheaper than SRAM.
- **SRAM:** Static RAM, retains data, until powered off.

ROM Read Only Memory

Read Only Memory, is non-volatile and is more like a permanent storage for information. It also stores the **bootstrap loader** program, to load and start the operating system when computer is turned on. **PROM** (Programmable ROM), **EPROM** (Erasable PROM) and **EEPROM** (Electrically Erasable PROM) are some commonly used ROMs.

Cache Memory

The **cache memory** is used to store program data that is currently being executed in the CPU. Whenever the CPU needs to access memory, it first checks the cache memory. If the data is not found in cache memory then the CPU moves onto the main memory.

The Memory Hierarchy

Registers

These are small, ultra-fast memory locations within the CPU used to hold data that is being processed. Registers are crucial for executing instructions efficiently.

Tertiary and Offline Memory

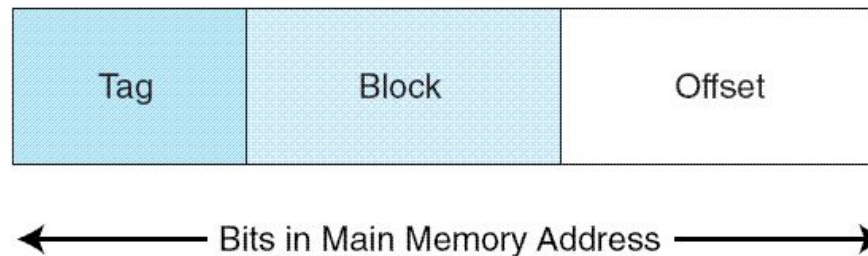
- **Tertiary memory** refers to storage devices used for backups and archives, like magnetic tapes.
- **Offline memory** is storage that is not directly accessible by the computer (e.g., external hard drives, optical discs) but data can be retrieved when connected.

Cache Memory

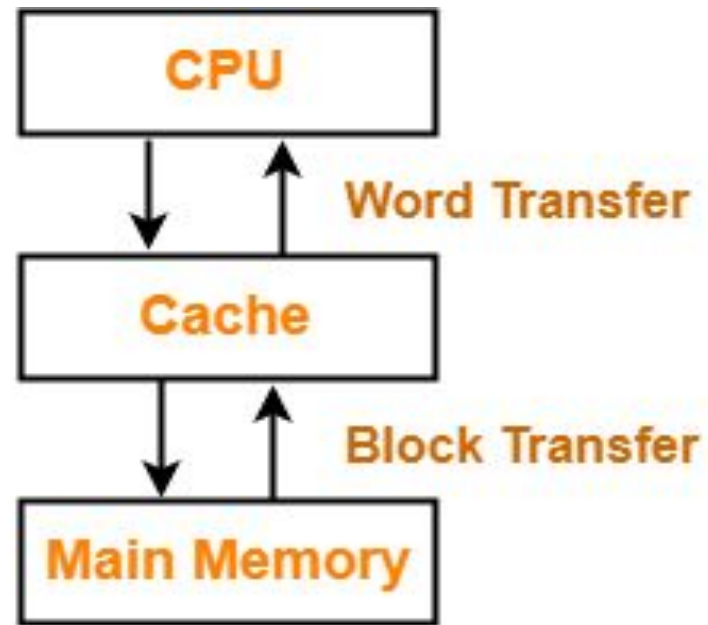
- The purpose of cache memory is to speed up accesses by storing recently used data closer to the CPU, instead of storing it in main memory.
- Although cache is much smaller than main memory, its access time is a fraction of that of main memory.
- Unlike main memory, which is accessed by address, cache is typically accessed by content; hence, it is often called *content addressable memory*.
- Because of this, a single large cache memory isn't always desirable-- it takes longer to search.

Cache Memory

- The “content” that is addressed in content addressable cache memory is a subset of the bits of a main memory address called a *field*.
 - Many blocks of main memory map to a single block of cache. A *tag* field in the cache block distinguishes one cached memory block from another.
 - A *valid bit* indicates whether the cache block is being used.
 - An *offset field* points to the desired data in the block.



- Data is transferred in the form of words between the cache memory and the CPU.
- Data is transferred in the form of blocks or pages between the cache memory and the main memory.



Cache and Main Memory

Execution Of Program

- Whenever any program has to be executed, it is first loaded in the main memory.
- The portion of the program that is mostly probably going to be executed in the near future is kept in the cache memory.
- This allows CPU to access the most probable portion at a faster speed.

Step-01: Whenever CPU requires any word of memory, it is first searched in the CPU registers.

Now, there are two cases possible-

Case-01: If the required word is found in the CPU registers, it is read from there.

Case-02: If the required word is not found in the CPU registers, Step-02 is followed.

Step-02:

- When the required word is not found in the CPU registers, it is searched in the cache memory.
- Tag directory of the cache memory is used to search whether the required word is present in the cache memory or not.

Now, there are two cases possible-

Case-01:

- If the required word is found in the cache memory, the word is delivered to the CPU.
- This is known as **Cache hit**.

Case-02:

- If the required word is not found in the cache memory, Step-03 is followed.
- This is known as **Cache miss**.

Step-03:

- When the required word is not found in the cache memory, it is searched in the main memory.
- Page Table is used to determine whether the required page is present in the main memory or not.

Now, there are two cases possible-

Case-01:

If the page containing the required word is found in the main memory,

- The page is mapped from the main memory to the cache memory.
- This mapping is performed using cache mapping techniques.
- Then, the required word is delivered to the CPU.

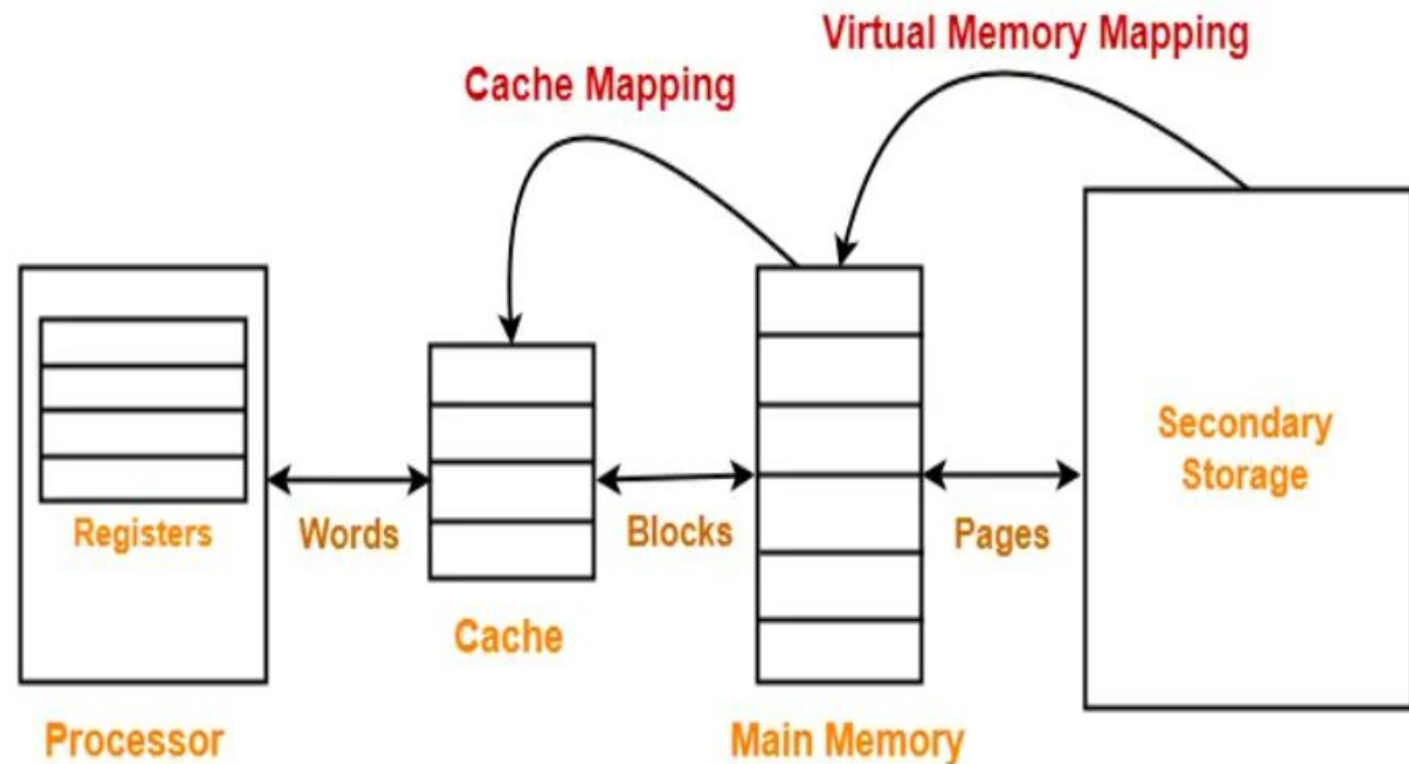
Case-02:

If the page containing the required word is not found in the main memory,

- A page fault occurs.
- The page containing the required word is mapped from the secondary memory to the main memory.
- Then, the page is mapped from the main memory to the cache memory.
- 83 ● Then, the required word is delivered to the CPU.

Mapping Techniques

- Cache mapping defines how a block from the main memory is mapped to the cache memory in case of a cache miss. OR
- Cache mapping is a technique by which the contents of main memory are brought into the cache memory.
- Main memory is divided into equal size partitions called as **blocks** or **frames**.
- Cache memory is divided into partitions having same size as that of blocks called as **lines**.
- During cache mapping, block of main memory is simply copied to the cache and the block is not actually brought from the main memory.



Mapping Techniques

Cache Mapping Techniques-

1. Direct Mapping
2. Fully Associative Mapping
3. K-way Set Associative Mapping

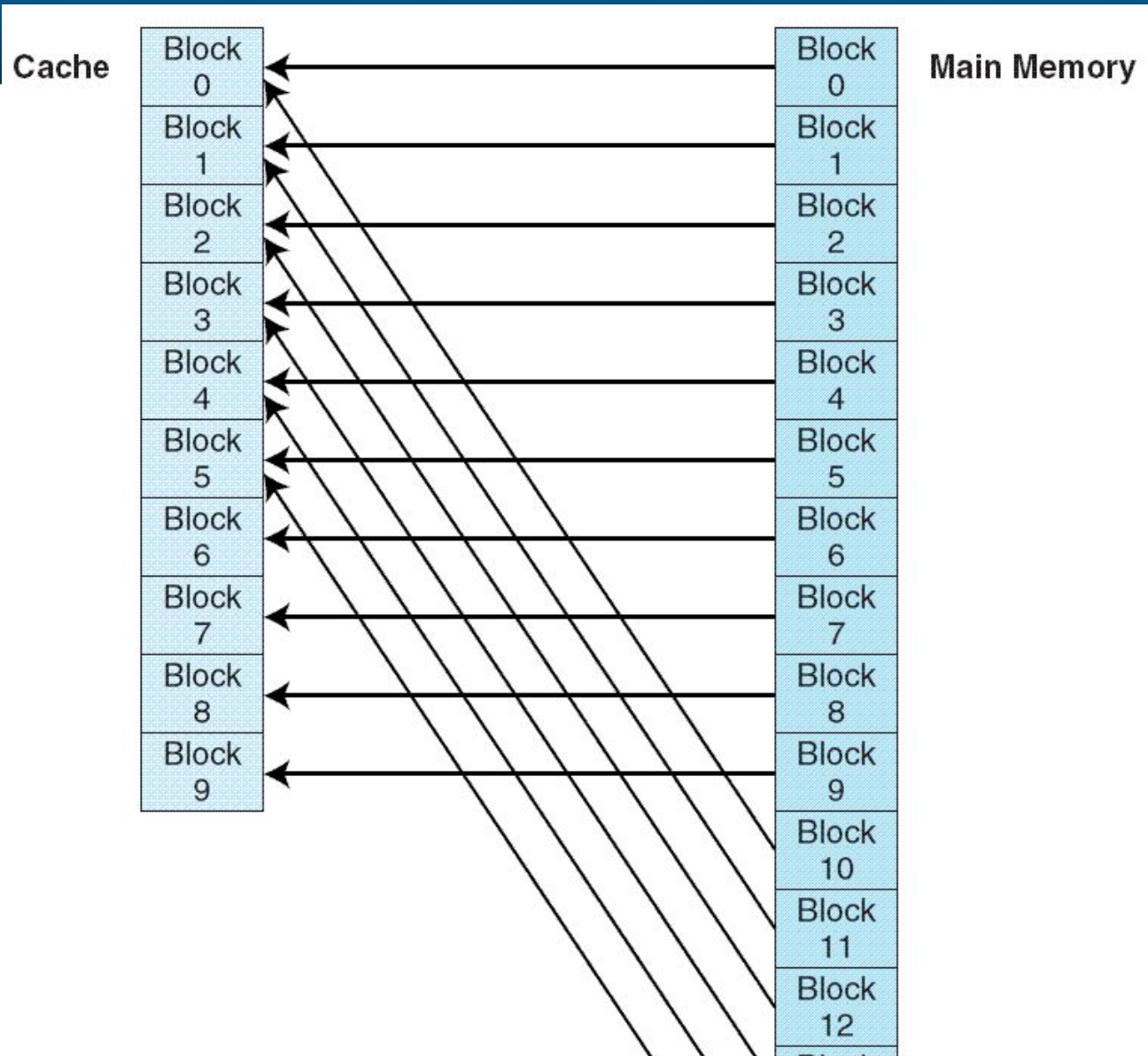
Mapping Techniques

1. Direct Mapping-

- The simplest cache mapping scheme is *direct mapped cache*.
- In direct mapping, each block of main memory maps to exactly one specific cache line.
- The mapping is done using a modulus operation.
- In a direct mapped cache consisting of N blocks of cache, block X of main memory maps to cache block $Y = X \bmod N$.
- Thus, if we have 10 blocks of cache, block 7 of cache may hold blocks 7, 17, 27, 37, . . . of main memory.

Mapping Techniques

- With direct mapped cache consisting of N blocks of cache, block X of main memory maps to cache block $Y = X \bmod N$.



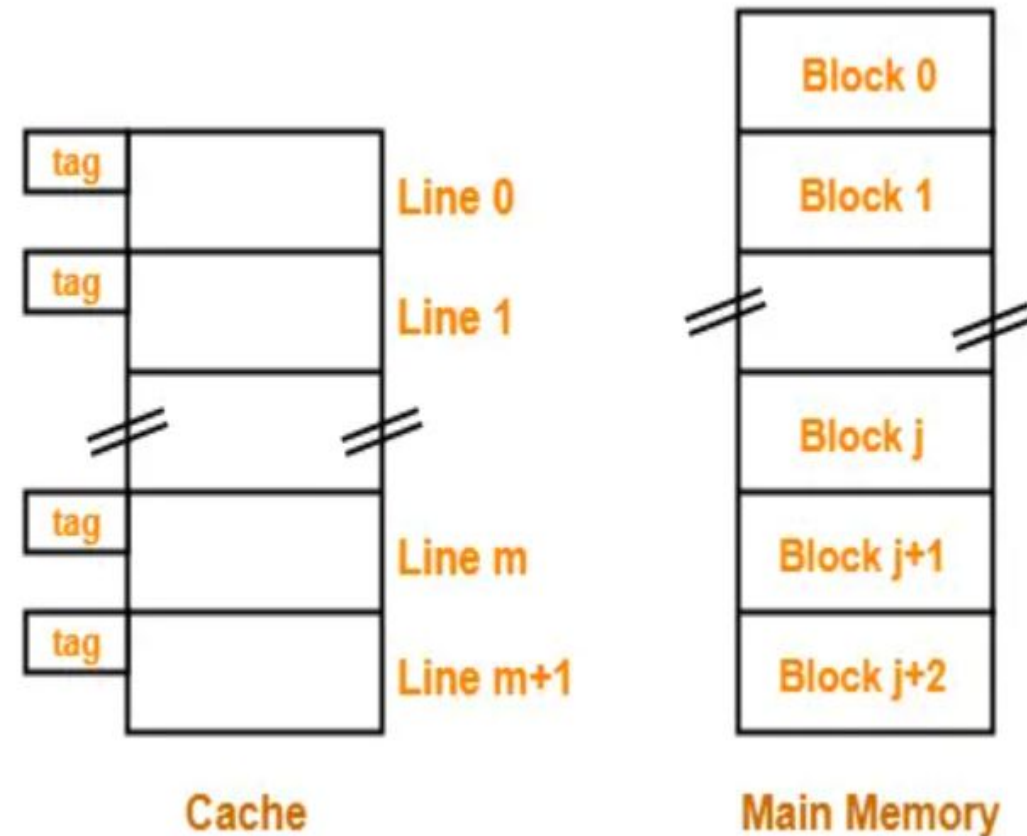
Mapping Techniques

2. Fully Associative Mapping-

- A block of main memory can map to any line of the cache that is freely available at that moment.
- This makes fully associative mapping more flexible than direct mapping.

Here,

- All the lines of cache are freely available.
- Thus, any block of main memory can map to any line of the cache.
- Had all the cache lines been occupied, then one of the existing blocks will have to be replaced.



Mapping Techniques

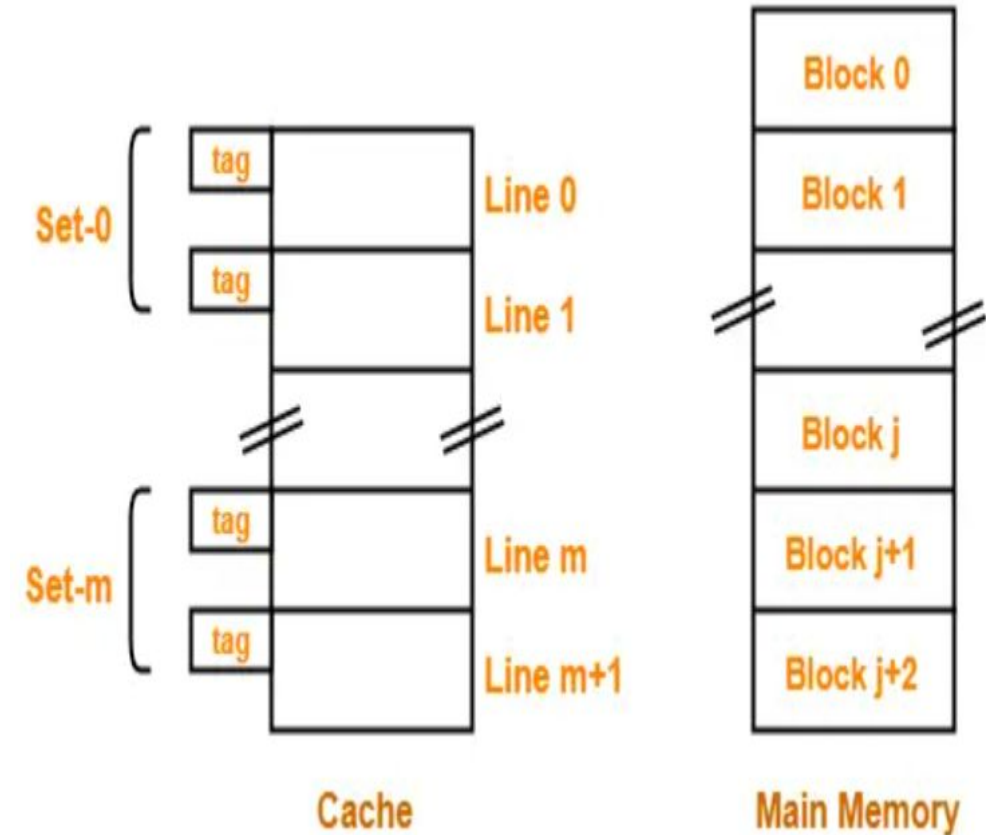
3. K-way Set Associative Mapping-

In k-way set associative mapping,

- Cache lines are grouped into sets where each set contains k number of lines.
- A particular block of main memory can map to only one particular set of the cache.
- However, within that set, the memory block can map any cache line that is freely available.
- The set of the cache to which a particular block of the main memory can map is given by-

Cache set number

$$= (\text{Main Memory Block Address}) \text{ Modulo } (\text{Number of sets in Cache})$$



- Cache size = 8 lines (Line 0 to Line 7)
- Cache is divided into **4 sets**, with 2 lines per set (2-way set-associative).
- Main memory has 32 blocks (Block 0 to Block 31).

Each block maps to a specific set using the formula: $Set\ Number = (Block\ Number) \bmod (Number\ of\ Sets)$

$$= 30 \bmod 4 = 2$$

Replacement Policies

FIFO - First In, First Out:

FIFO is a cache replacement policy where the oldest items are removed first when the cache reaches its maximum capacity. Imagine the cache as a queue, and the first item that was added to the cache is the first one to be evicted when the cache is full. This strategy ensures that the data that has been in the cache the longest is replaced, making room for new or more frequently accessed data.

LIFO - Last In, First Out:

LIFO is another cache replacement policy, but instead of removing the oldest items, it removes the most recently added items when the cache is full. This means that the last item added to the cache will be the first one to be evicted when the cache is at its limit.

LRU - Least Recently Used:

LRU is a popular cache replacement policy that removes the least recently used items from the cache when it is full. This policy assumes that the data that has not been accessed for the longest time is less likely to be accessed again soon.

MRU - Most Recently Used:

MRU is the opposite of LRU. It removes the most recently used items from the cache when it is full, assuming that the most recently used data is more likely to be accessed again soon.

LFU - Least Frequently Used:

LFU is a cache replacement policy that removes the least frequently used items from the cache when it is full. It relies on a counter associated with each item in the cache, tracking the number of times the data has been accessed.

Reference Questions

1. What are the key differences between RISC and CISC architectures?
2. Explain the advantages of RISC over CISC in terms of instruction execution.
3. Why does CISC architecture require more complex hardware than RISC?
4. What is the function of the ALU in a CPU?
5. Describe the role of the control unit in instruction execution.
6. What is the difference between hardwired and microprogrammed control units?
7. List advantages and disadvantages of microprogrammed control.
8. In what type of systems is hardwired control preferred?
9. Why is microprogrammed control easier to modify?
10. What is pipelining in computer architecture?
11. Explain the term "pipeline hazard" and its types.
12. How does pipelining improve CPU performance?
13. Explain memory organization with its types.
14. Explain the hierarchy of memory from fastest to slowest.
15. What is cache memory and why is it needed?
16. Explain direct mapping, associative mapping, and set-associative mapping.
17. What are the common cache replacement policies?
18. Explain following- a)Auxiliary Memory b) Volatile Memory c) Non Volatile memory d)virtual memory
19. Explain stepwise execution of program with respect to cache memory.
20. ₉₃ Explain cache mapping and their types.

Reference Questions

21. Which type of control unit (hardwired or microprogrammed) would be more suitable for implementing a new complex instruction set, and why?
22. In a pipelined processor, what happens if an instruction causes a branch to a different location?
23. Can a RISC processor execute fewer instructions than a CISC processor and still be faster? Explain.
24. In which condition would set-associative cache mapping be more beneficial than direct mapping?
25. What type of memory (volatile or non-volatile) would be used in cache, and why?